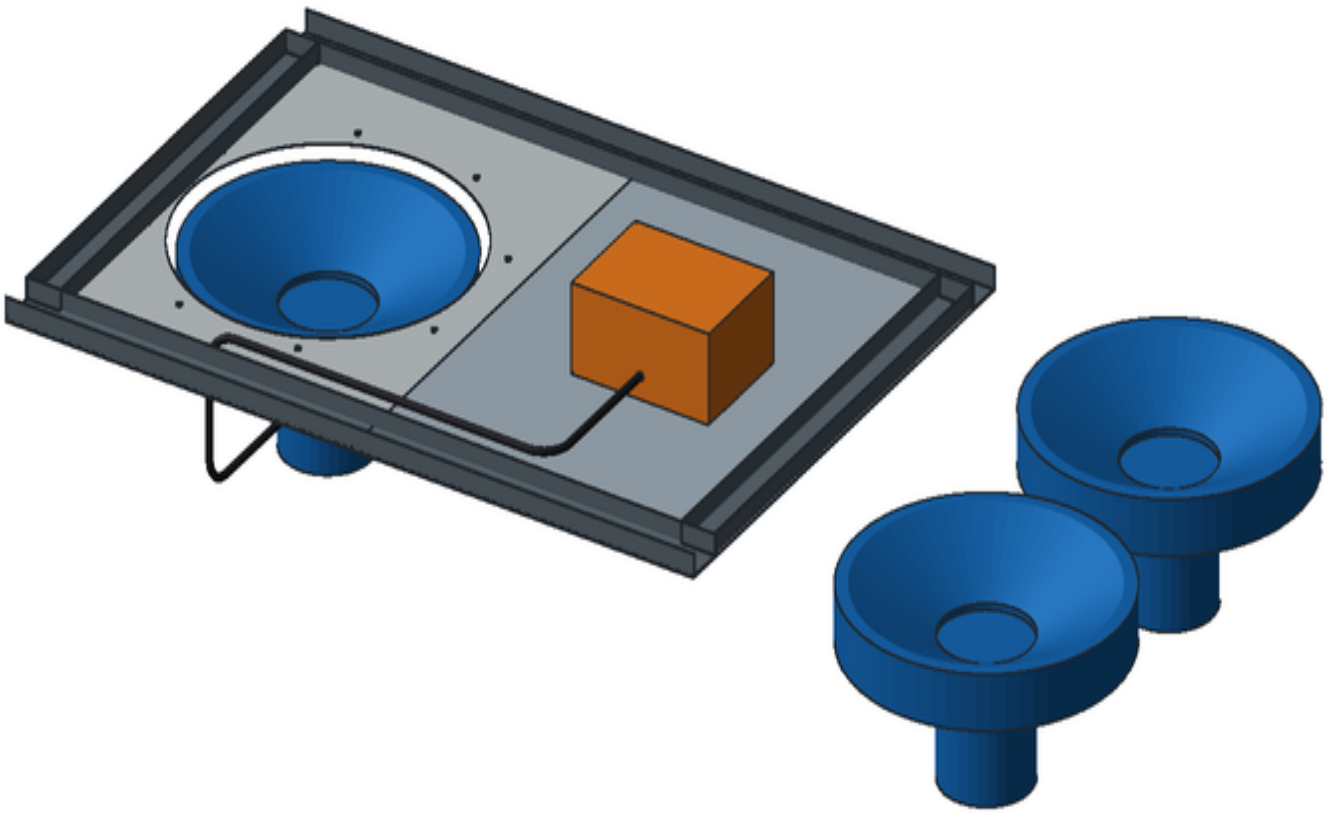
QPAC.Sculptor

Building STEP files from scratch

A function-by-function walkthrough for version 1.1.1, with a tested example and a rendered result for every function.



About this walkthrough

This walkthrough builds STEP files from nothing, one function at a time. Each page explains a single function: what it is for, its parameters, a complete example, and a picture of the solid that example produces.

Every example in this document was compiled and run against QPAC.Sculptor 1.1.1 to make its picture, so the code and the pictures always match. The STEP files the examples write were also opened with OpenCascade, an independent CAD kernel, to confirm they are valid solids.

Lengths are in inches unless a page says otherwise. Angles are in degrees.

How a model is put together

```
StepFile                     the document: one .step file (Units, Author, Description)
 +- parts       List<StepPart>    components in the CAD tree, placed where drawn
 |    +- shapes  List<StepShape>  the solids: boxes, extrusions, sweeps ...
 +- instances   List<StepInstance> extra placed copies of a part
```

Each shape is turned into a closed solid made of flat polygon faces (a Polyhedron). The STEP writer and the WPF preview both draw from that one solid. In the pictures in this document, solids are drawn in their STEP colours, and grey stands for shapes with no colour. Dark lines mark edges where faces meet at more than 30 degrees.

Contents

Part 1 - From an empty project to a STEP file 5

Step 0 - Create a project and add the package . . . 5
Step 1 - Create the file: StepFile . . . 5
Step 2 - Add a part: AddPart / StepPart . . . 6
Step 3 - Add a shape: part.Add . . . 6
Step 4 - Write the file: WriteStep . . . 7
Step 5 - Check it: Validate . . . 7

Part 2 - Primitive shapes 9

Rectangle - a box . . . 9
Rectangle.FromCorners - a box between two corners . . . 10
Rectangle.WithFillet / WithChamfer - rounded or bevelled box . . . 10
Rectangle.Oriented - turn a box onto axes (legacy) . . . 11
Cylinder - a rod between two points . . . 12
Cone - cones and frustums . . . 13
Sphere . . . 13
Torus - rings and gaskets . . . 14
Tube - pipe, sleeves, spacers . . . 15

Part 3 - Extrusions 16

new Extrusion - the sketch plane . . . 16
Extrusion.AlongZ and SetProfile . . . 17
AddCut / AddCircularCut / AddRectangularCut / AddSlotCut - through-holes . . . 17

AddBoltCircle / AddHoleGrid - hole patterns . . . 18
AddPocket / AddCircularPocket / AddRectangularPocket - blind cuts . . . 19
Shell - hollow it into a tray or tube . . . 20
AddPocketGrid - ribbed, lightened plates . . . 21
FilletCorners - round the corners of the outline . . . 21
FilletEdges / ChamferEdges - round or bevel the cap edges . . . 22
WithDraft - taper the walls . . . 23

Part 4 - Profiles: 2D outlines **24**

Profiles.Rectangle / RoundedRectangle . . . 24
Profiles.Circle / Ellipse . . . 24
Profiles.RegularPolygon . . . 25
Profiles.Slot . . . 26
Profiles.Arc . . . 26
Profiles.Polygon . . . 27
Profiles.Channel - C / U section . . . 27
Profiles.Angle - L section . . . 28
Profiles.ZSection . . . 28
Profiles.ThickPolyline - any sheet-metal section . . . 29
Profiles.RectangularTubeInner . . . 29
Profiles.Fillet / FilletCorners . . . 30
Profiles.Chamfer . . . 30
Profiles.Offset . . . 31
Translate / Rotate / Scale / MirrorU / MirrorV / Area . . . 31

Part 5 - Revolves, sweeps and lofts **33**

Revolve - spin a section around an axis . . . 33
Sweep and Sweep.Rod - carry a section along a path . . . 34
Loft and Loft.SquareToRound - blend between two sections . . . 35
MeshSolid - bring your own polyhedron . . . 35

Part 6 - Moving things **37**

RotateAbout / Translate - order matters . . . 37
MirrorAcross / TransformBy . . . 38
Moving a whole part . . . 38
Transform - building transforms yourself . . . 39

Part 7 - Colours, assemblies and instances **40**

StepColor and WithColor . . . 40
AddInstance - one part, many copies . . . 40

Part 8 - Checking, measuring and handling errors **42**

When an error is reported . . . 42

SkipInvalidShapes - export what you can . . . 42

ComputeVolume / Bounds / ToPolyhedron . . . 43

Part 9 - Previewing in WPF (QPAC.Sculptor.Wpf) **44**

SteptoVisual3D.Build . . . 44

SculptorRenderOptions . . . 44

Picking a part with the mouse . . . 45

BuildModel / BuildPart / ToMesh and threading . . . 45

Part 10 - Putting it together **46**

Worked example: fan-wall cell . . . 46

Troubleshooting **48**

Review findings and suggestions **49**

Part 1 - From an empty project to a STEP file

These five steps are all you need to produce a valid STEP file. Later parts go through every shape and feature in detail.

Step 0 - Create a project and add the package

QPAC.Sculptor is a normal NuGet package published to the private qpacsystems GitHub Packages feed. Any .NET project can use it: console tools, web services, WPF apps or class libraries.

COMMANDS

```
dotnet new console -n MyStepTool
cd MyStepTool
dotnet nuget add source "https://nuget.pkg.github.com/qpacsystems/index.json" ^
    --name QPACSource --username <github-user> --password <token> --store-password-in-clear-text
dotnet add package QPAC.Sculptor --version 1.1.1
```

- The token is a GitHub personal access token (classic) with read:packages. On Q-PAC machines the QPACSource feed is usually configured already; skip that line if it is.
- Reference an exact version, or 1.1.*. A reference of 1.0.* never picks up 1.1.0.
- The core package targets netstandard2.0 and net7.0, so it works from .NET Framework 4.7.2 up through .NET 8 and later. Only the separate QPAC.Sculptor.Wpf package needs Windows.
- Add 'using QPAC.Sculptor;' to your file. Everything is in that namespace except StepFile, which is in the global namespace so that older compiled consumers keep working.

Step 1 - Create the file: StepFile

A StepFile is the whole document: one .step file. It holds the parts, the unit every coordinate is measured in, and a few header fields.

```
new StepFile { Units, Author, Organization, Description }
```

PARAMETERS

`Units`	LengthUnit the model is drawn in. Default Inch. Also Millimeter, Centimeter, Meter, Foot.
`Author / Organization`	Written into the STEP header. Organization defaults to "Q-PAC".
`Description`	Header description. Defaults to empty; when it is empty the header uses "<product name> - boundary representation".

EXAMPLE

```
var file = new StepFile { Description = "My first model", Author = "R. Griggs" };
Console.WriteLine($"Drawing in {file.Units}, {file.parts.Count} parts so far");
```

OUTPUT

```
Drawing in Inch, 0 parts so far
```

- Decide the unit first and draw everything in it. Converting to another unit on export is a separate option (step 4).

Step 2 - Add a part: AddPart / StepPart

A part is one component in the CAD assembly tree, such as a door, a fan or a panel. It holds one or more shapes. Each shape stays a separate solid; the shapes are not fused together.

```
file.AddPart(new StepPart(name[, color]))   or   file.AddPart(name)
```

PARAMETERS

`name`	Component name shown in the CAD tree.
`color`	Optional StepColor for every shape in the part.
`Type`	Optional free-text field for your own use (the WPF preview can colour by it).

EXAMPLE

```
var file = new StepFile();
var panel = file.AddPart(new StepPart("Side panel", StepColor.SheetMetal) { Type = "Panel" });
var bracket = file.AddPart("Bracket");
Console.WriteLine(string.Join(", ", file.parts.Select(p => p.Name)));
```

OUTPUT

```
Side panel, Bracket
```

- A part with no shapes is simply left out of the file.

Step 3 - Add a shape: part.Add

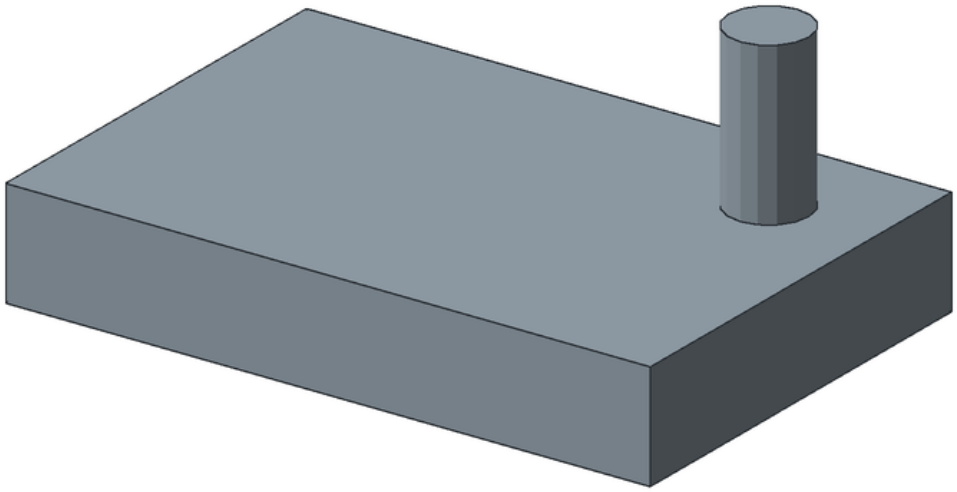
Shapes are the solids. part.Add returns the shape you pass it, so you can keep configuring it in the same statement.

```
part.Add(shape)  ->  the same shape
```

EXAMPLE

```
var file = new StepFile();
var part = file.AddPart(new StepPart("Block", StepColor.SheetMetal));
part.Add(new Rectangle("Base", 0, 0, 0.5, 6, 4, 1));          // 6 x 4 x 1 box
part.Add(new Cylinder("Peg", 2, 1, 1, 2, 1, 2.5, 0.375));     // peg standing on it
file.WriteStep("block.step", "Block");
```

RESULT



Step 4 - Write the file: WriteStep

Writes the model as an ISO 10303-21 (AP214) STEP file that SolidWorks, Inventor, Fusion, FreeCAD and other CAD tools open.

```
file.WriteStep(path, productName[, options])
Step.WriteStep(path, file, productName[, options])
Step.WriteStep(stream, file, productName[, options])
Step.ToStepString(file, productName[, options])
```

PARAMETERS

`productName`	Name of the root assembly. Blank becomes "Model".
`options.ExportUnits`	Convert every coordinate to another unit on the way out (null = keep Units).
`options.IncludeColors`	Write colours (default true).
`options.DefaultColor`	Colour for shapes with none (default: leave uncoloured).
`options.ValidateSolids`	Fully check every solid and throw on the first problem (default false).
`options.SkipInvalidShapes`	Leave out shapes that fail instead of throwing; see SkippedShapes.

EXAMPLE

```
var file = new StepFile();
file.AddPart("Block").Add(new Rectangle("b", 0, 0, 0, 2, 2, 2));

file.WriteStep("block-in.step", "Block");                                   // inches
file.WriteStep("block-mm.step", "Block",
    new StepWriteOptions { ExportUnits = LengthUnit.Millimeter });          // converted to mm
string text = Step.ToStepString(file, "Block");
Console.WriteLine(text.Split('\n')[2]);
```

OUTPUT

```
/* Length unit: INCH. Every coordinate in this file is in inches. STEP defines imperial units relative
to SI, so a MILLIMETRE unit also appears below only as the base of the INCH conversion. */
```

- Why does an inch file mention millimetres? STEP cannot define an inch on its own, only as "25.4 x MILLIMETRE". So an inch file contains a MILLIMETRE unit plus CONVERSION_BASED_UNIT('INCH', ...), and all geometry uses the inch unit. The comment on the third line of the file (printed above) says this. CAD tools read the file as inches.
- A StepWriteOptions object collects SkippedShapes across writes. Use a new one for each export.

Step 5 - Check it: Validate

Validate() builds every shape and lists anything that would not make a clean solid, without throwing. Run it before you export, and in your tests.

```
file.Validate()  ->  List<string>
```

EXAMPLE

```
var file = new StepFile();
var door = file.AddPart("Door");
door.Add(Extrusion.AlongZ("Skin", 0, 0.05, Profiles.Rectangle(0, 0, 10, 10))
    .AddCircularCut(20, 0, 1));                 // hole is outside the 10 x 10 skin
foreach (var problem in file.Validate()) Console.WriteLine(problem);
```

OUTPUT

```
Door / Skin: Skin: cut 0 runs outside the outline or too close to its edge.
```

- An empty list means every shape will export as a closed, consistent solid. Solids that pass through themselves, and shapes that overlap each other, are not checked.
- Validate() also reports zero-size shapes (a zero-length cylinder, or an extrusion whose profile was never set) as "only 0 faces". WriteStep silently skips those. If you create zero-size shapes on purpose, filter them out or ignore those messages.

Part 2 - Primitive shapes

Primitives are solids defined by a few numbers. Every shape also accepts the transform methods in Part 6 and WithColor(...).

Curved surfaces are made of flat facets. The sides / segments parameters control how many there are: more facets look smoother but make bigger files.

Rectangle - a box

A box given by its centre and its full size along X, Y and Z.

```
new Rectangle(name, cx, cy, cz, sx, sy, sz)
```

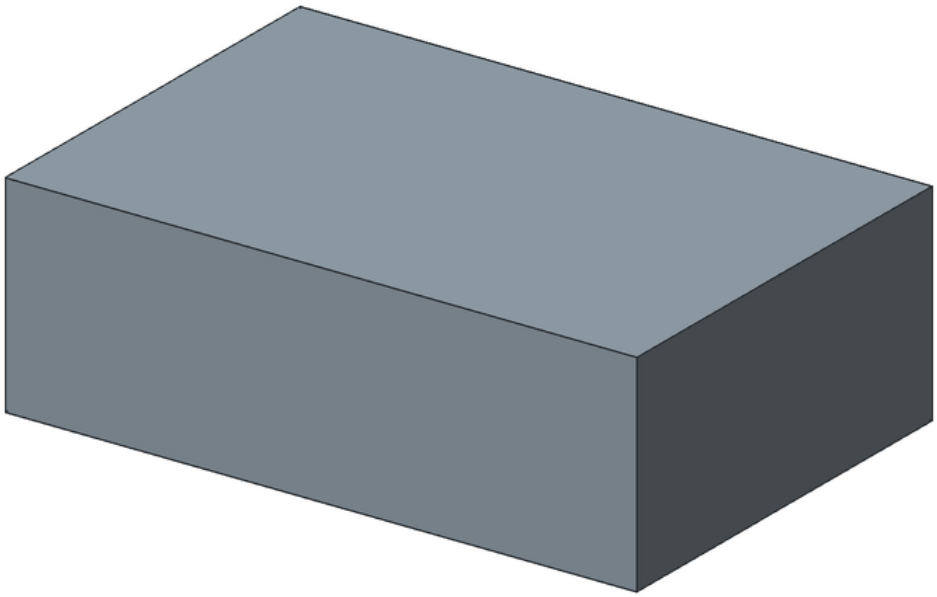
PARAMETERS

`cx, cy, cz`	Centre of the box.
`sx, sy, sz`	Full size along X, Y, Z.

EXAMPLE

```
var file = new StepFile();
file.AddPart(new StepPart("Box", StepColor.SheetMetal))
    .Add(new Rectangle("Box", 0, 0, 1, 6, 4, 2));
file.WriteStep("box.step", "Box");
```

RESULT



Rectangle.FromCorners - a box between two corners

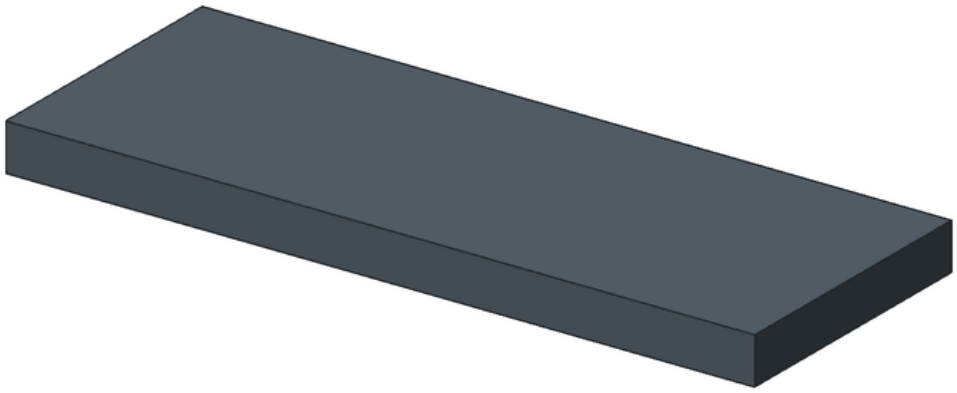
The same box, specified by two opposite corners instead of a centre and sizes. Corner order doesn't matter.

```
Rectangle.FromCorners(name, (x1, y1, z1), (x2, y2, z2))
```

EXAMPLE

```
var file = new StepFile();
file.AddPart(new StepPart("Slab", StepColor.Channel))
    .Add(Rectangle.FromCorners("Slab", (0, 0, 0), (8, 3, 0.5)));
file.WriteStep("slab.step", "Slab");
```

RESULT



Rectangle.WithFillet / WithChamfer - rounded or bevelled box

Rounds (or bevels) all twelve edges of the box at once.

```
box.WithFillet(radius, segments = 6)
box.WithChamfer(distance)
```

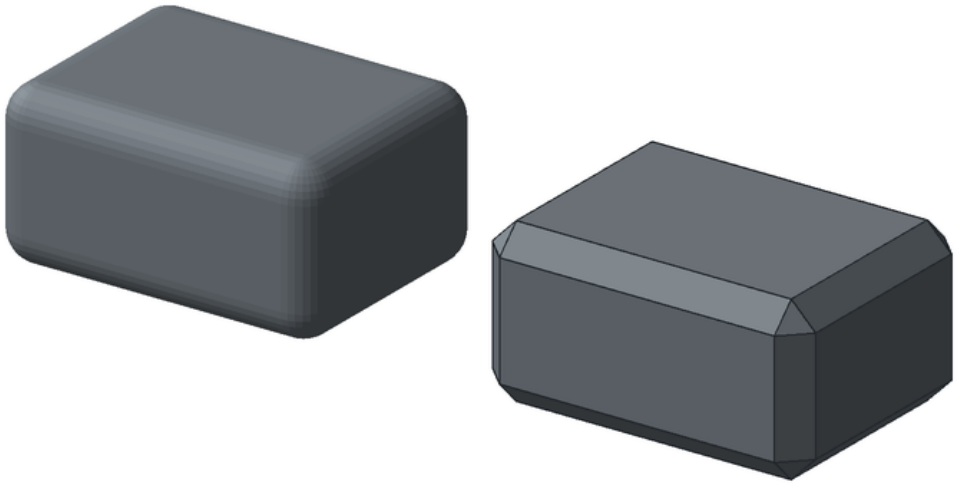
PARAMETERS

`radius / distance`	Size of the round or bevel. Clamped to half the smallest box size.
`segments`	Facets around each rounded edge.

EXAMPLE

```
var file = new StepFile();
var part = file.AddPart(new StepPart("Blocks", StepColor.Mount));
part.Add(new Rectangle("Filleted", 0, 0, 1, 4, 3, 2).WithFillet(0.4, 8));
part.Add(new Rectangle("Chamfered", 6, 0, 1, 4, 3, 2).WithChamfer(0.3));
file.WriteStep("blocks.step", "Blocks");
```

RESULT



- A radius equal to half the smallest size rounds that direction completely; for a cube that gives a ball-like solid.

Rectangle.Oriented - turn a box onto axes (legacy)

Turns the box about its centre so its local X/Y/Z line up with three perpendicular unit vectors that you supply. This comes from 1.0. For new code, RotateAbout (Part 6) is easier.

```
box.Oriented(xAxis, yAxis, zAxis)
```

EXAMPLE

```
var file = new StepFile();
double c = Math.Sqrt(0.5);
file.AddPart(new StepPart("Turned", StepColor.Fan))
    .Add(new Rectangle("Turned", 0, 0, 0, 6, 1, 1).Oriented((c, c, 0), (-c, c, 0), (0, 0, 1)));
file.WriteStep("turned.step", "Turned");
```

RESULT



- The three vectors must be unit length and at right angles to each other. Nothing checks this for you.

Cylinder - a rod between two points

A round rod between two end-centre points, in any direction.

```
new Cylinder(name, x1, y1, z1, x2, y2, z2, radius, sides = 16)
new Cylinder(name, start, end, radius, sides = 16)
.WithFillet(radius, segments = 6)
```

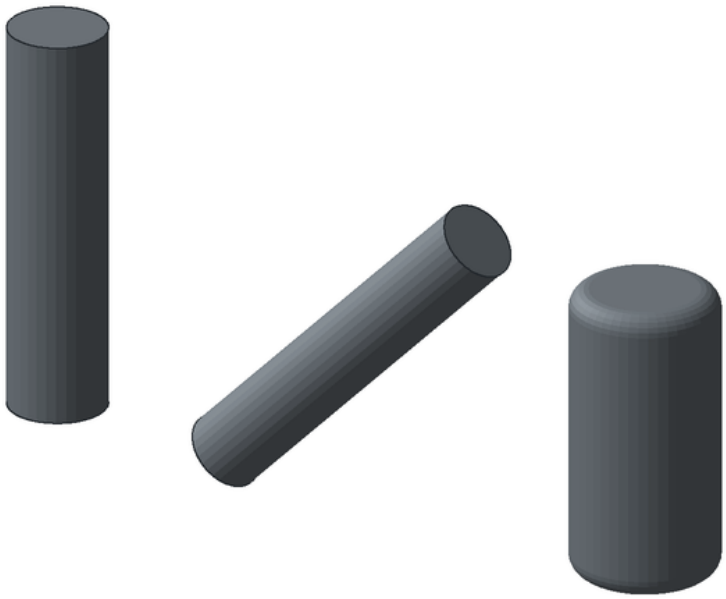
PARAMETERS

`start / end`	End-centre points.
`radius`	Rod radius.
`sides`	Facets around the rod.

EXAMPLE

```
var file = new StepFile();
var part = file.AddPart(new StepPart("Rods", StepColor.Mount));
part.Add(new Cylinder("Upright", 0, 0, 0, 0, 0, 4, 0.5, 32));
part.Add(new Cylinder("Diagonal", (2, 0, 0), (5, 0, 3), 0.4, 32));
part.Add(new Cylinder("Rounded", (7, 0, 0), (7, 0, 3), 0.75, 48).WithFillet(0.2));
file.WriteStep("rods.step", "Rods");
```

RESULT



- A zero-length cylinder is skipped on export (no error).

Cone - cones and frustums

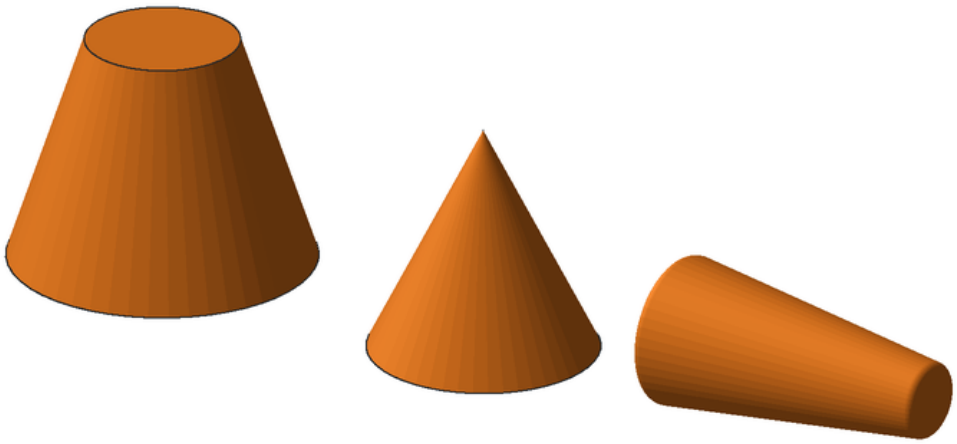
A cone or truncated cone. The constructor stands it on +Z; Cone.Between points it anywhere.

```
new Cone(name, cx, cy, cz, height, bottomRadius, topRadius, sides = 32)
Cone.Between(name, baseCentre, topCentre, bottomRadius, topRadius, sides = 32)
.WithFillet(radius, segments = 6)
```

EXAMPLE

```
var file = new StepFile();
var part = file.AddPart(new StepPart("Cones", StepColor.Controller));
part.Add(new Cone("Frustum", 0, 0, 0, 3, 2, 1, 48));
part.Add(new Cone("Point", 5, 0, 0, 3, 1.5, 0, 48));
part.Add(Cone.Between("Sideways", (8, 0, 1), (12, 0, 1), 1, 0.5, 48).WithFillet(0.1));
file.WriteStep("cones.step", "Cones");
```

RESULT



- A radius of 0 is raised to 0.0001, so a pointed cone has a tiny flat tip (same as version 1.0).

Sphere

A ball.

```
new Sphere(name, cx, cy, cz, radius, segments = 32, rings = 16)
```

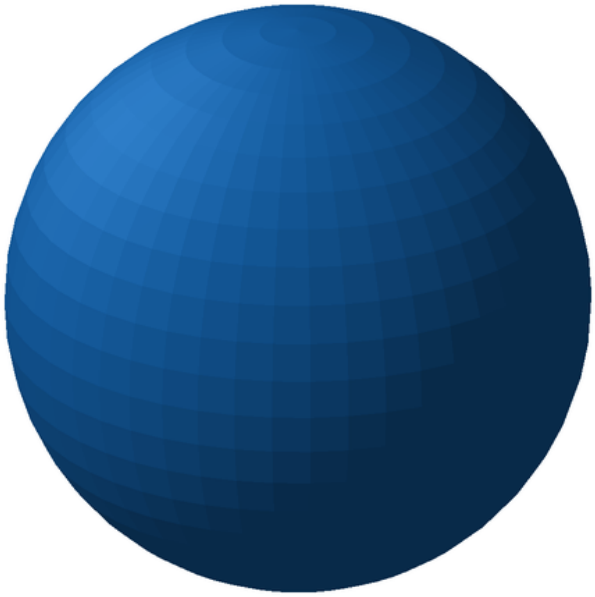
PARAMETERS

segments	Facets around the equator.
rings	Bands from pole to pole.

EXAMPLE

```
var file = new StepFile();
file.AddPart(new StepPart("Ball", StepColor.Fan))
    .Add(new Sphere("Ball", 0, 0, 2, 2, 48, 24));
file.WriteStep("ball.step", "Ball");
```

RESULT



Torus - rings and gaskets

A doughnut around an axis. majorRadius is the distance to the centre of the tube, and minorRadius is the tube's own radius.

```
new Torus(name, centre, axis, majorRadius, minorRadius, segments = 48, tubeSegments = 16)
```

EXAMPLE

```
var file = new StepFile();
file.AddPart(new StepPart("Gasket", StepColor.Rubber))
    .Add(new Torus("O-ring", (0, 0, 0.5), (0, 0, 1), 3, 0.5));
file.WriteStep("oring.step", "Gasket");
```

RESULT



- Requires 0 < minorRadius <= majorRadius.

Tube - pipe, sleeves, spacers

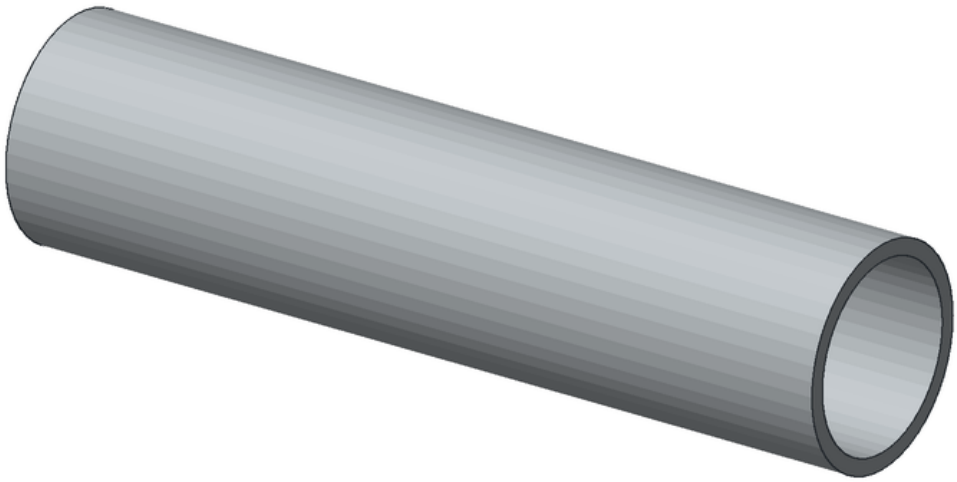
A hollow round tube between two points. An innerRadius of 0 gives a solid rod.

```
new Tube(name, start, end, outerRadius, innerRadius, sides = 32)
```

EXAMPLE

```
var file = new StepFile();
file.AddPart(new StepPart("Pipe", StepColor.Galvanized))
    .Add(new Tube("Pipe", (0, 0, 1), (8, 0, 1), 1, 0.85, 48));
file.WriteStep("pipe.step", "Pipe");
```

RESULT



- innerRadius must be smaller than outerRadius.

Part 3 - Extrusions

An extrusion takes a closed 2D outline (a profile) and pushes it a distance to make a solid. Most sheet-metal and machined parts are extrusions. This part covers the sketch plane, outlines, holes, pockets, hollowing and edge finishing.

Terms used below. The front face is the face in the sketch plane, at the origin. The back face is the one Depth away. With Extrusion.AlongZ(name, z, ...) the front face is the bottom, at height z.

new Extrusion - the sketch plane

Defines where the 2D sketch lives in 3D space and how far to push it.

```
new Extrusion(name, origin, u, v, depth)
```

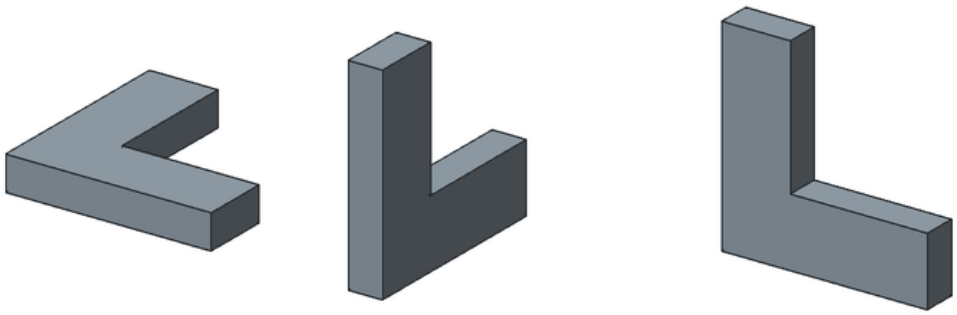
PARAMETERS

`origin`	3D point where profile coordinate (0, 0) sits.
`u, v`	3D directions of the sketch's U and V axes. They are normalised, and v is made perpendicular to u.
`depth`	Distance pushed along W = U x V. Negative pushes the other way.

EXAMPLE

```
var file = new StepFile();
var part = file.AddPart(new StepPart("Planes", StepColor.SheetMetal));
// Same L-shaped outline sketched on three planes:
var outline = Profiles.Polygon((0, 0), (3, 0), (3, 1), (1, 1), (1, 3), (0, 3));
part.Add(new Extrusion("On XY", (0, 0, 0), (1, 0, 0), (0, 1, 0), 0.5).SetProfile(outline));   // pushes +Z
part.Add(new Extrusion("On YZ", (5, 0, 0), (0, 1, 0), (0, 0, 1), 0.5).SetProfile(outline));   // pushes +X
part.Add(new Extrusion("On XZ", (8, 4, 0), (1, 0, 0), (0, 0, 1), 0.5).SetProfile(outline));   // pushes -Y
file.WriteStep("planes.step", "Planes");
```

RESULT



- Work out the push direction as U x V. For example, (1,0,0) x (0,0,1) = (0,-1,0), so the XZ sketch goes toward -Y.

Extrusion.AlongZ and SetProfile

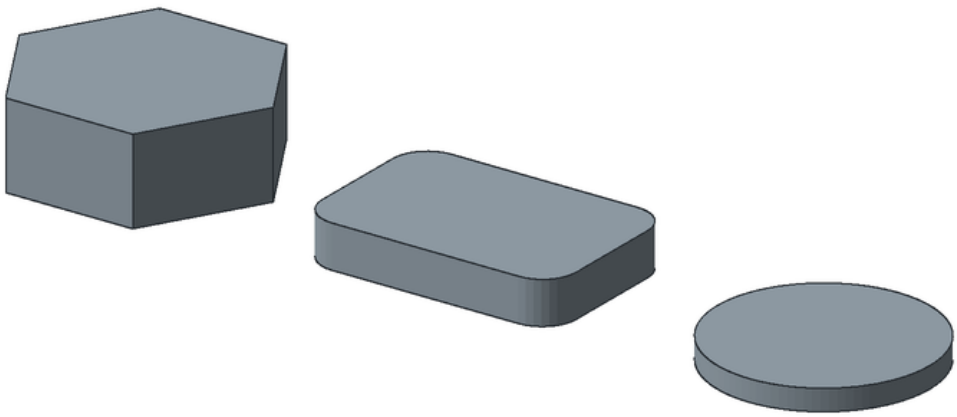
AlongZ is shorthand for the most common case: sketch on the XY plane at height z and push up +Z. SetProfile replaces the outline with any closed loop, usually one from Profiles (Part 4).

```
Extrusion.AlongZ(name, z, depth, profile)
.SetProfile(loop)   .AddRectangle(cu, cv, w, h, cornerRadius = 0)
.AddPolygon(points...)   .AddCircle(cu, cv, radius, sides = 256)
```

EXAMPLE

```
var file = new StepFile();
var part = file.AddPart(new StepPart("Outlines", StepColor.SheetMetal));
part.Add(Extrusion.AlongZ("Hexagon", 0, 1, Profiles.RegularPolygon(0, 0, 1.5, 6)));
part.Add(new Extrusion("Rounded", (0, 0, 0), (1, 0, 0), (0, 1, 0), 0.5).AddRectangle(4, 0, 3, 2, 0.4));
part.Add(new Extrusion("Disc", (0, 0, 0), (1, 0, 0), (0, 1, 0), 0.25).AddCircle(8, 0, 1.25, 96));
file.WriteStep("outlines.step", "Outlines");
```

RESULT



- AddCircle sets the outline to a circle; it does not add a hole. The name comes from 1.0. Use AddCircularCut for a hole.
- Loops can wind either way. The outline must not cross itself.

AddCut / AddCircularCut / AddRectangularCut / AddSlotCut - through-holes

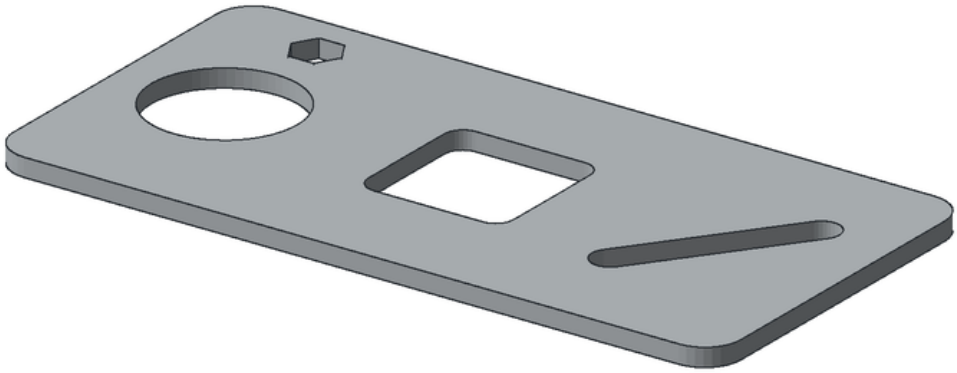
Each cut removes material through the whole thickness.

```
.AddCut(loop)
.AddCircularCut(cu, cv, radius, sides = 256)
.AddRectangularCut(cu, cv, w, h, cornerRadius = 0)
.AddSlotCut(u1, v1, u2, v2, width)
```

EXAMPLE

```
var file = new StepFile();
file.AddPart(new StepPart("Bracket", StepColor.Galvanized))
    .Add(Extrusion.AlongZ("Bracket", 0, 0.25, Profiles.RoundedRectangle(0, 0, 10, 5, 0.5))
        .AddCircularCut(-3.5, 0, 1, 64)
        .AddRectangularCut(0, 0, 2, 2, 0.25)
        .AddSlotCut(2.5, -1, 4, 1, 0.5)
        .AddCut(Profiles.RegularPolygon(-3.5, 1.8, 0.35, 6)));
file.WriteStep("bracket.step", "Bracket");
```

RESULT



- Cuts must lie fully inside the outline and must not overlap each other. Breaking this is reported by Validate() and throws at WriteStep or in the preview, not when you call AddCut.

AddBoltCircle / AddHoleGrid - hole patterns

Evenly spaced round holes: on a circle, or in rows and columns.

```
.AddBoltCircle(cu, cv, patternRadius, count, holeRadius, startDegrees = 0, sides = 32)
.AddHoleGrid(u0, v0, columns, rows, pitchU, pitchV, holeRadius, sides = 32)
```

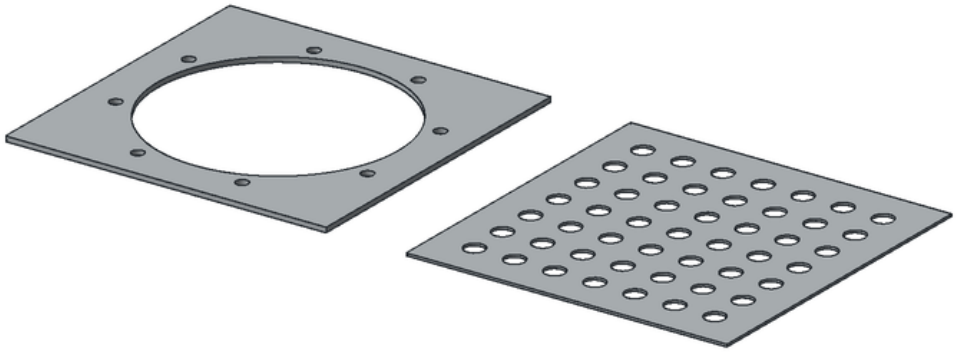
PARAMETERS

`startDegrees`	Angle of the first hole, measured from +U.
`u0, v0`	Centre of the first (bottom-left) hole of the grid.

EXAMPLE

```
var file = new StepFile();
var part = file.AddPart(new StepPart("Plates", StepColor.Galvanized));
part.Add(Extrusion.AlongZ("Fan plate", 0, 0.125, Profiles.Rectangle(0, 0, 8, 8))
    .AddCircularCut(0, 0, 3, 128)
    .AddBoltCircle(0, 0, 3.4, 8, 0.15, 22.5));
part.Add(Extrusion.AlongZ("Perforated", 0, 0.0625, Profiles.Rectangle(10, 0, 8, 8))
    .AddHoleGrid(7, -3, 7, 7, 1, 1, 0.25, 24));
file.WriteStep("patterns.step", "Plates");
```

RESULT



AddPocket / AddCircularPocket / AddRectangularPocket - blind cuts

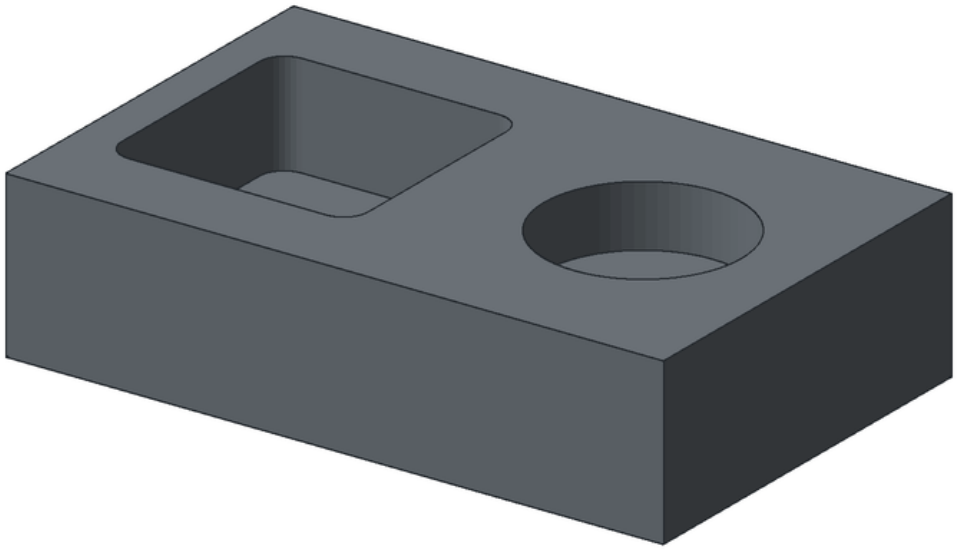
A pocket removes material from one face down to a depth and leaves a floor. Pockets come from the front face by default, or from the back with fromBack: true.

```
.AddPocket(loop, depth, fromBack = false)
.AddCircularPocket(cu, cv, radius, depth, fromBack = false, sides = 64)
.AddRectangularPocket(cu, cv, w, h, depth, cornerRadius = 0, fromBack = false)
```

EXAMPLE

```
var file = new StepFile();
file.AddPart(new StepPart("Block", StepColor.Mount))
    .Add(Extrusion.AlongZ("Block", 0, 2, Profiles.Rectangle(0, 0, 8, 5))
        .AddRectangularPocket(-2, 0, 3, 3, 1.25, 0.3, fromBack: true)   // opens on top
        .AddCircularPocket(2, 0, 1.2, 0.75, fromBack: true)
        .AddCircularCut(2, 0, 0.4, 32));                                // through the pocket floor
file.WriteStep("pockets.step", "Block");
```

RESULT



- Depth must be greater than 0 and less than the extrusion depth. Use a cut to go all the way through.
- Pockets on the same face must not overlap. A front pocket and a back pocket may overlap if their depths together leave material between them.
- A cut drawn inside a pocket goes through the pocket floor. A cut that crosses a pocket's edge is an error.
- Pocket walls are always straight. Draft (WithDraft) does not tilt them.

Shell - hollow it into a tray or tube

Offsets the outline inward by wallThickness and pockets it down to floorThickness, turning a solid into a tray. With floorThickness 0 it cuts all the way through instead, leaving an open-ended tube or duct.

```
.Shell(wallThickness, floorThickness, openFront = true)
```

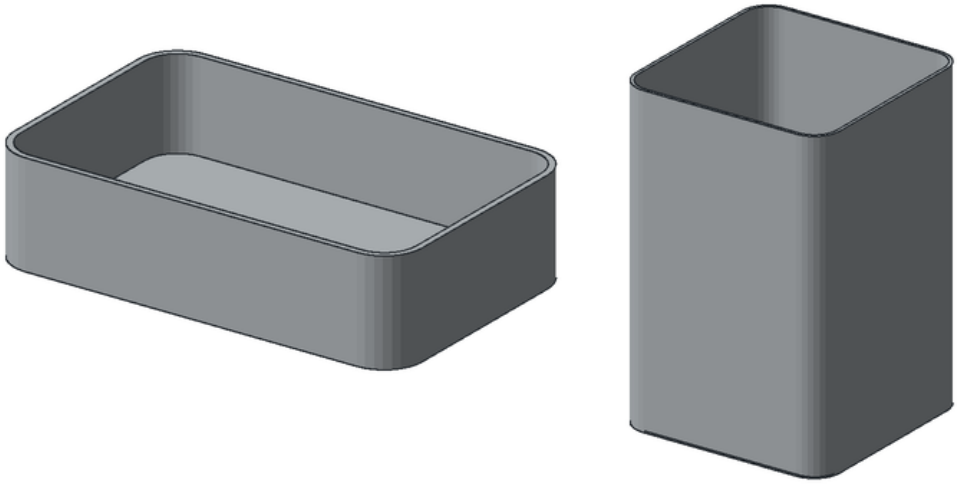
PARAMETERS

openFront	true opens the front face (the bottom with AlongZ); false opens the back (the top).

EXAMPLE

```
var file = new StepFile();
var part = file.AddPart(new StepPart("Shells", StepColor.Galvanized));
part.Add(Extrusion.AlongZ("Tray", 0, 2, Profiles.RoundedRectangle(0, 0, 8, 5, 0.75))
    .Shell(0.125, 0.25, openFront: false)                    // floor at the bottom, open on top
    .AddCircularCut(2.5, 0, 0.4, 32));                       // drain hole in the floor
part.Add(Extrusion.AlongZ("Duct", 0, 6, Profiles.RoundedRectangle(10, 0, 4, 4, 0.5))
    .Shell(0.0625, 0));                                      // open tube
file.WriteStep("shells.step", "Shells");
```

RESULT



- Shell reads the outline when you call it, so set the profile first.
- The wall must be thinner than any rounded corner's radius, and less than half the narrowest section of the outline. Otherwise the inward offset turns inside out, and Shell itself throws straight away.
- Shell does not follow draft. On a drafted part, a thin wall can break through the tapered side, and nothing detects it.

AddPocketGrid - ribbed, lightened plates

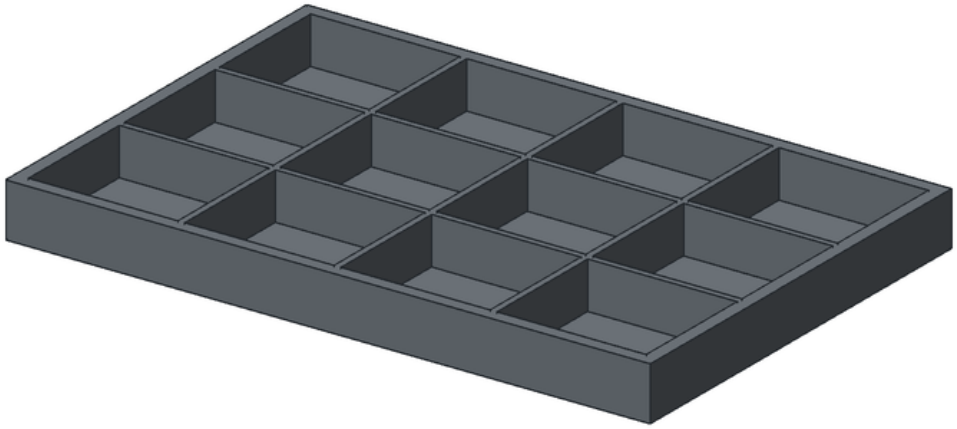
Like Shell, but leaves a grid of stiffening ribs standing in the hollow.

```
.AddPocketGrid(wallThickness, floorThickness, ribThickness, columns, rows, openFront = true)
```

EXAMPLE

```
var file = new StepFile();
file.AddPart(new StepPart("Ribbed", StepColor.Mount))
    .Add(Extrusion.AlongZ("Ribbed plate", 0, 1, Profiles.Rectangle(0, 0, 12, 8))
        .AddPocketGrid(0.25, 0.125, 0.125, 4, 3, openFront: false));
file.WriteStep("ribbed.step", "Ribbed");
```

RESULT



- The hollowed interior must be convex: rectangles, circles and rounded rectangles work, an L-shape does not (it throws immediately).
- floorThickness must be greater than 0. Unlike Shell, 0 is an error, reported when the shape is built.
- Set the profile before calling it.

FilletCorners - round the corners of the outline

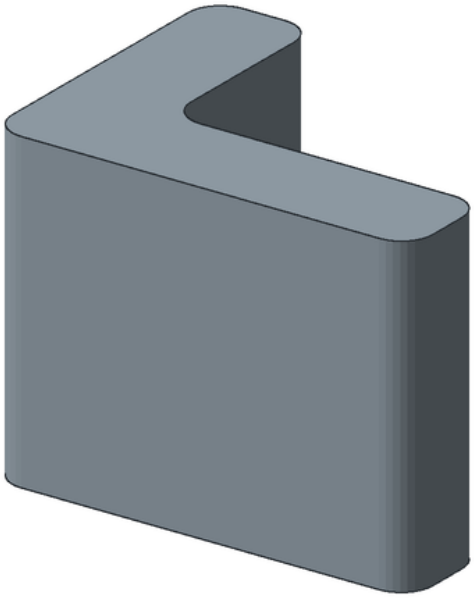
Rounds every corner of the outline, which rounds the edges that run along the length of the extrusion.

```
.FilletCorners(radius, segmentsPerCorner = 8)
```

EXAMPLE

```
var file = new StepFile();
file.AddPart(new StepPart("Rounded", StepColor.SheetMetal))
    .Add(Extrusion.AlongZ("Bar", 0, 3, Profiles.Polygon((0, 0), (4, 0), (4, 1), (1, 1), (1, 3), (0, 3)))
        .FilletCorners(0.3));
file.WriteStep("rounded.step", "Rounded");
```

RESULT



- It changes the outline only, not cuts. Call it before Shell or AddPocketGrid if the inside should be rounded too.

FilletEdges / ChamferEdges - round or bevel the cap edges

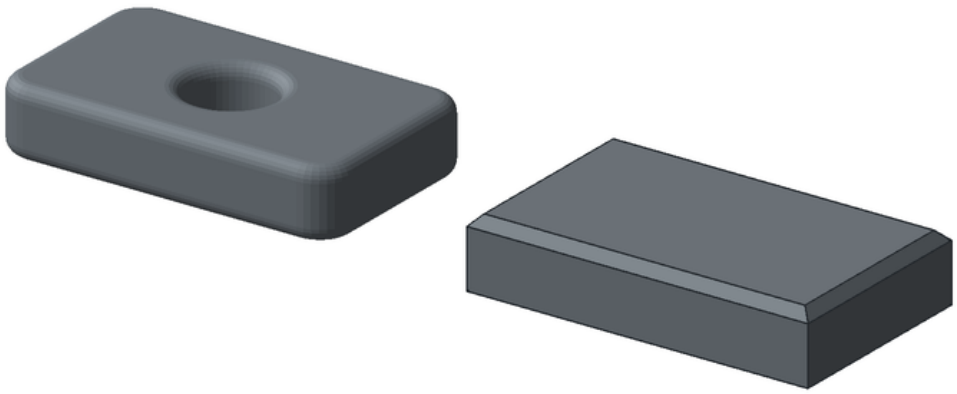
Rounds or bevels the edges around the front and/or back faces. This also rounds the lips of through-holes that run from face to face.

```
.FilletEdges(radius, segments = 6, front = true, back = true)
.ChamferEdges(distance, front = true, back = true)
```

EXAMPLE

```
var file = new StepFile();
var part = file.AddPart(new StepPart("Edges", StepColor.Mount));
part.Add(Extrusion.AlongZ("Filleted", 0, 1, Profiles.RoundedRectangle(0, 0, 5, 3, 0.5))
    .AddCircularCut(0, 0, 0.6, 48)
    .FilletEdges(0.2, 6));
part.Add(Extrusion.AlongZ("Chamfered", 0, 1, Profiles.Rectangle(7, 0, 5, 3))
    .ChamferEdges(0.15, front: false));              // top (back-face) edges only
file.WriteStep("edges.step", "Edges");
```

RESULT



- The front and back sizes together must not exceed the depth. The radius must also be smaller than any rounded corner's radius, and less than half the narrowest section, or the shape throws a clear error. Sharp corners are fine.
- Pocket edges, and holes that start or end at a pocket floor, are not rounded.
- For a fully rounded block, use FilletCorners(r) and FilletEdges(r) together, or Rectangle.WithFillet.

WithDraft - taper the walls

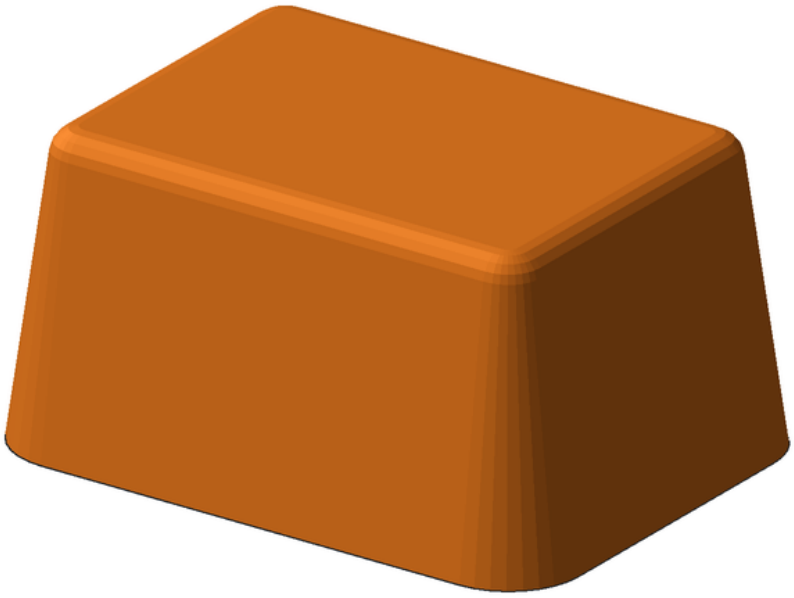
Tilts the outer walls inward by an angle as they go toward the back face, like a moulded or cast part. Through-holes taper too and get larger toward the back.

```
.WithDraft(degrees)
```

EXAMPLE

```
var file = new StepFile();
file.AddPart(new StepPart("Boss", StepColor.Controller))
    .Add(Extrusion.AlongZ("Boss", 0, 2, Profiles.RoundedRectangle(0, 0, 4, 3, 0.5))
        .WithDraft(8)
        .FilletEdges(0.15, 4, front: false));
file.WriteStep("boss.step", "Boss");
```

RESULT



- Pockets and Shell interiors are not drafted (see the Shell page).

Part 4 - Profiles: 2D outlines

Profiles builds closed 2D loops of (U, V) points. Use them as outlines and cuts for extrusions, as sections for sweeps, and as the ends of lofts. Every Profiles shape comes back counter-clockwise. The plots show the loop with a dot at each vertex; the cross marks (0, 0).

Profiles.Rectangle / RoundedRectangle

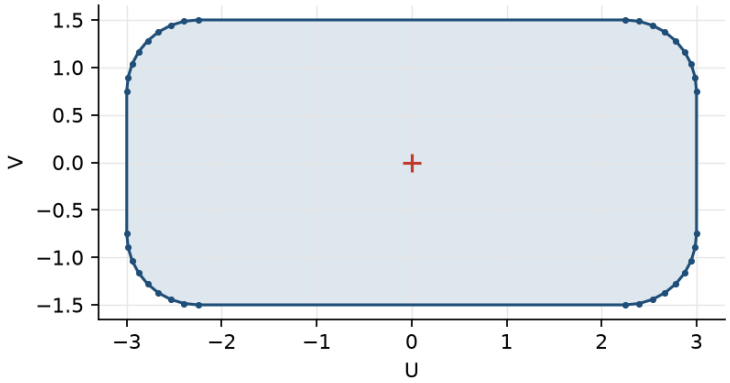
Rectangles centred on (cu, cv). The corner radius is clamped to fit.

```
Profiles.Rectangle(cu, cv, w, h)
Profiles.RoundedRectangle(cu, cv, w, h, radius, segmentsPerCorner = 8)
```

EXAMPLE

```
var profile = Profiles.RoundedRectangle(0, 0, 6, 3, 0.75);
```

RESULT



Profiles.Circle / Ellipse

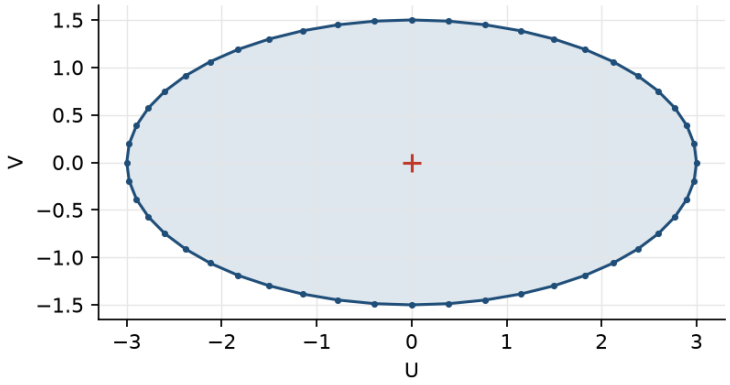
Round outlines with sides straight segments.

```
Profiles.Circle(cu, cv, radius, sides = 64)
Profiles.Ellipse(cu, cv, radiusU, radiusV, sides = 64)
```

EXAMPLE

```
var profile = Profiles.Ellipse(0, 0, 3, 1.5, 48);
```

RESULT



Profiles.RegularPolygon

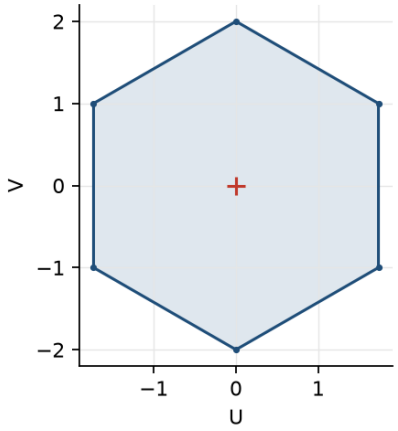
Hexagons, octagons and so on. The radius is measured to the corners.

```
Profiles.RegularPolygon(cu, cv, radius, sides, rotationDegrees = 0)
```

EXAMPLE

```
var profile = Profiles.RegularPolygon(0, 0, 2, 6, 30);
```

RESULT



Profiles.Slot

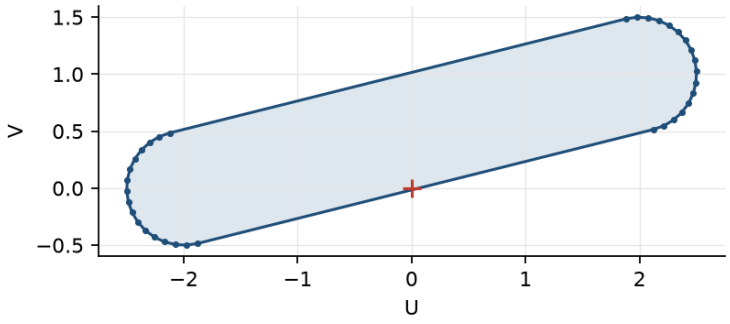
An obround slot between two end-centre points.

```
Profiles.Slot(u1, v1, u2, v2, width, segmentsPerEnd = 16)
```

EXAMPLE

```
var profile = Profiles.Slot(-2, 0, 2, 1, 1);
```

RESULT



Profiles.Arc

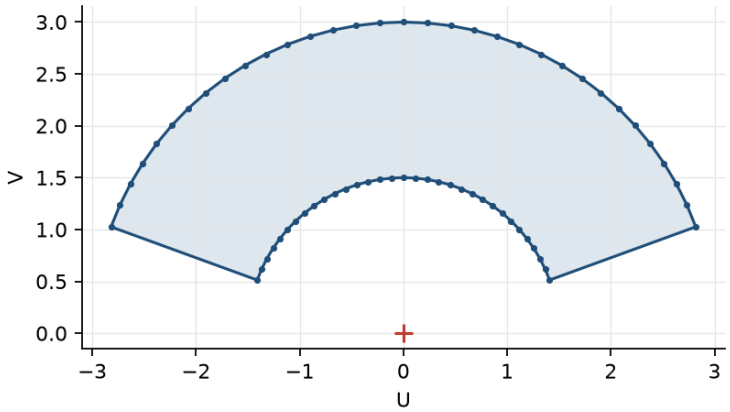
A ring sector. An innerRadius of 0 gives a pie slice.

```
Profiles.Arc(cu, cv, innerRadius, outerRadius, startDegrees, endDegrees, segments = 32)
```

EXAMPLE

```
var profile = Profiles.Arc(0, 0, 1.5, 3, 20, 160);
```

RESULT



Profiles.Polygon

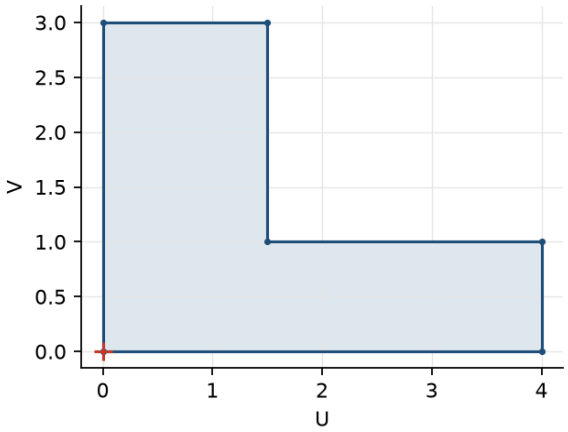
Any outline you describe point by point, in either winding direction.

```
Profiles.Polygon(points...)
```

EXAMPLE

```
var profile = Profiles.Polygon((0, 0), (4, 0), (4, 1), (1.5, 1), (1.5, 3), (0, 3));
```

RESULT



Profiles.Channel - C / U section

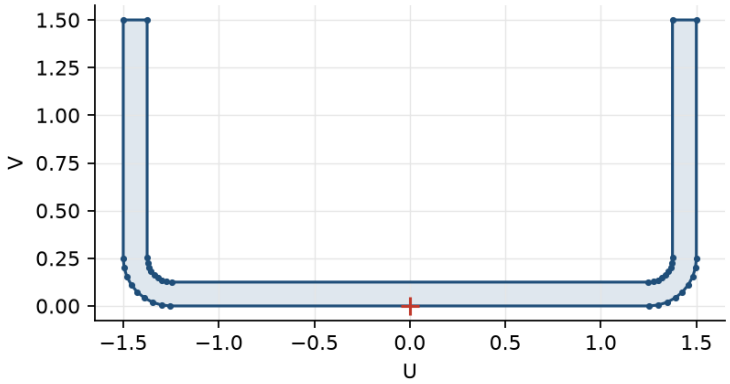
A formed channel opening toward +V. width and flangeHeight are outside dimensions, and the origin is the centre of the outside of the web.

```
Profiles.Channel(width, flangeHeight, thickness, insideBendRadius = 0, bendSegments = 8)
```

EXAMPLE

```
var profile = Profiles.Channel(3, 1.5, 0.125, 0.125);
```

RESULT



Profiles.Angle - L section

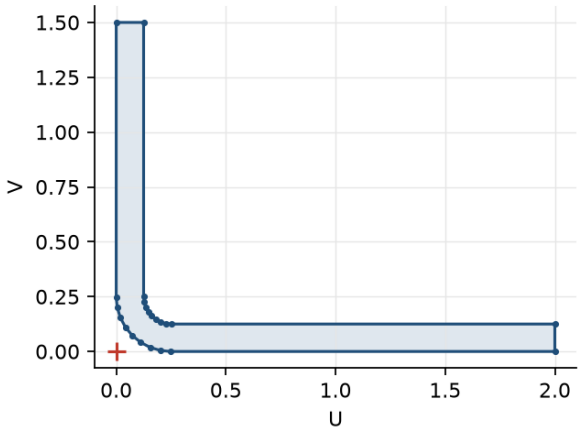
A formed angle with the outside corner at the origin and the legs along +U and +V.

```
Profiles.Angle(legU, legV, thickness, insideBendRadius = 0, bendSegments = 8)
```

EXAMPLE

```
var profile = Profiles.Angle(2, 1.5, 0.125, 0.125);
```

RESULT



Profiles.ZSection

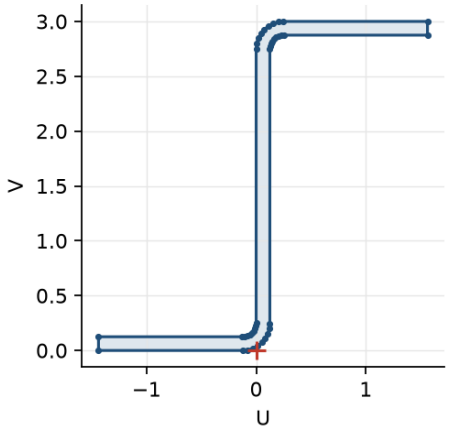
A Z section: bottom flange toward -U, web up +V, top flange toward +U. The web's outside face is at U = 0 and it spans U = 0 to U = thickness. The bottom flange tip is at U = -flange + thickness/2 and the top flange tip at U = flange + thickness/2.

```
Profiles.ZSection(flange, height, thickness, insideBendRadius = 0, bendSegments = 8)
```

EXAMPLE

```
var profile = Profiles.ZSection(1.5, 3, 0.125, 0.125);
```

RESULT



Profiles.ThickPolyline - any sheet-metal section

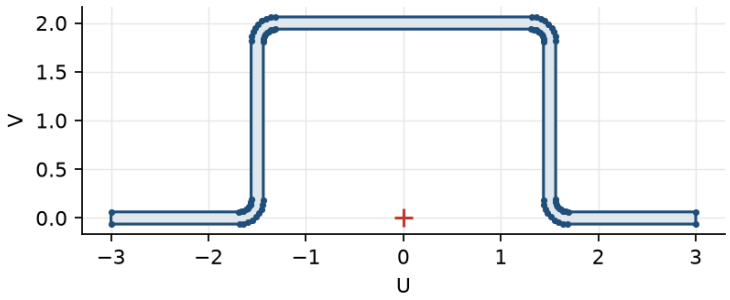
Thickens an open centreline into a constant-thickness section and rounds each bend. Channel, Angle and ZSection are built this way. Use it for hat sections, J-channels or any custom form.

```
Profiles.ThickPolyline(centreline, thickness, insideBendRadius = 0, bendSegments = 8)
```

EXAMPLE

```
var hat = new (double U, double V)[] { (-3, 0), (-1.5, 0), (-1.5, 2), (1.5, 2), (1.5, 0), (3, 0) };
var profile = Profiles.ThickPolyline(hat, 0.125, 0.125);
```

RESULT



- Thickness is centred on the line. Bend radius is the inside radius.

Profiles.RectangularTubeInner

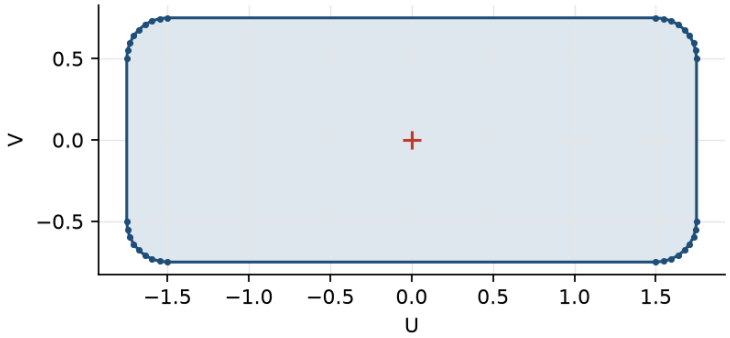
The inside loop of a rectangular tube. Pair it with a RoundedRectangle outline of the same size, as in the extrusion below.

```
Profiles.RectangularTubeInner(cu, cv, w, h, wall, outerRadius = 0, segmentsPerCorner = 8)
```

EXAMPLE

```
var profile = Profiles.RectangularTubeInner(0, 0, 4, 2, 0.25, 0.5);
```

RESULT



- Full use: Extrusion.AlongZ("Tube", 0, 36, Profiles.RoundedRectangle(0, 0, 4, 2, 0.5)).AddCut(Profiles.RectangularTubeInner(0, 0, 4, 2, 0.25, 0.5))

Profiles.Fillet / FilletCorners

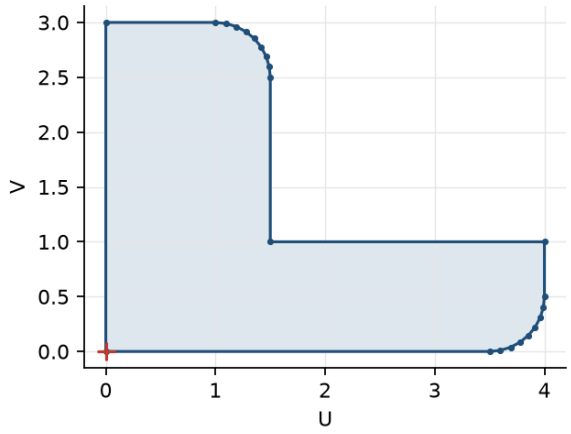
Rounds every corner, or only the corners whose vertex numbers you list (starting at 0).

```
Profiles.Fillet(loop, radius, segmentsPerCorner = 8)
Profiles.FilletCorners(loop, radius, segmentsPerCorner, cornerIndices...)
```

EXAMPLE

```
var l = Profiles.Polygon((0, 0), (4, 0), (4, 1), (1.5, 1), (1.5, 3), (0, 3));
var profile = Profiles.FilletCorners(l, 0.5, 8, 1, 4);      // outer corner at (4,0) and top-right (1.5,3)
```

RESULT



Profiles.Chamfer

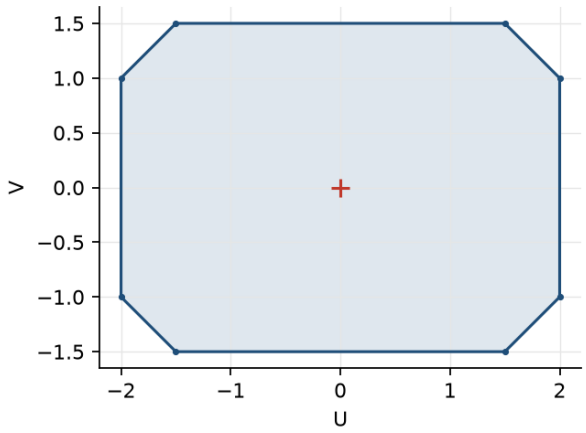
Cuts every corner back by distance along both edges.

```
Profiles.Chamfer(loop, distance)
```

EXAMPLE

```
var profile = Profiles.Chamfer(Profiles.Rectangle(0, 0, 4, 3), 0.5);
```

RESULT



Profiles.Offset

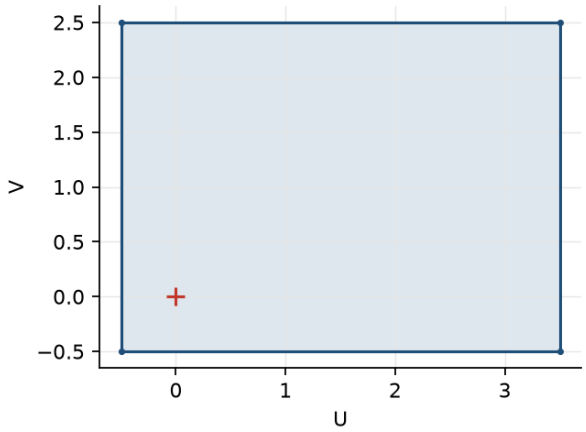
Grows the loop outward (positive distance) or shrinks it (negative), keeping sharp mitred corners. Useful for clearance cut-outs or a lip around an opening.

```
Profiles.Offset(loop, distance)
```

EXAMPLE

```
var profile = Profiles.Offset(Profiles.Polygon((0, 0), (3, 0), (3, 2), (0, 2)), 0.5);
```

RESULT



- Throws immediately if shrinking would turn the loop inside out.

Translate / Rotate / Scale / MirrorU / MirrorV / Area

Move a loop around in 2D before you use it. Area always returns a positive value.

```
Profiles.Translate(loop, du, dv)   Profiles.Rotate(loop, degrees, cu = 0, cv = 0)
Profiles.Scale(loop, factor, cu = 0, cv = 0)   Profiles.MirrorU(loop, axisU = 0)
Profiles.MirrorV(loop, axisV = 0)   Profiles.Area(loop)
```

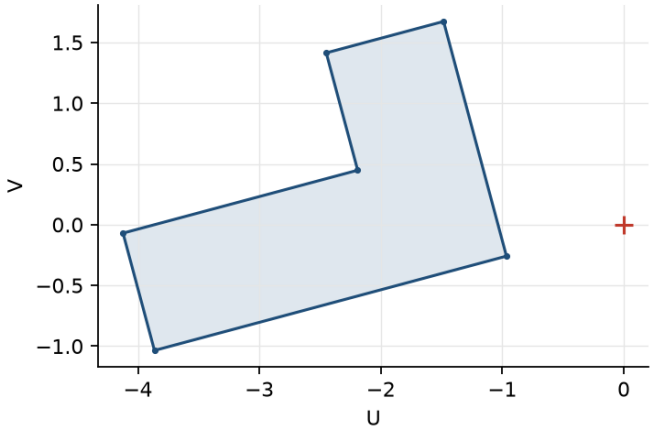
EXAMPLE

```
var l = Profiles.Polygon((0, 0), (3, 0), (3, 1), (1, 1), (1, 2), (0, 2));
var profile = Profiles.Rotate(Profiles.MirrorU(Profiles.Translate(l, 1, 0)), 15);
Console.WriteLine($"Area = {Profiles.Area(profile):F3}");
```

OUTPUT

```
Area = 4.000
```

RESULT



Part 5 - Revolves, sweeps and lofts

These three operations turn a 2D section into a solid by spinning it, carrying it along a path, or blending between two sections.

Revolve - spin a section around an axis

The lathe operation. The profile is a list of (R, Z) points: R is the distance from the axis (never negative) and Z is the position along it. A full revolve is closed; a partial one gets flat end faces.

```
new Revolve(name, origin, axis, profile, angleDegrees = 360, segments = 64)
  .StartDegrees   .ReferenceDirection   .FilletProfile(radius, segmentsPerCorner = 6, corners...)
```

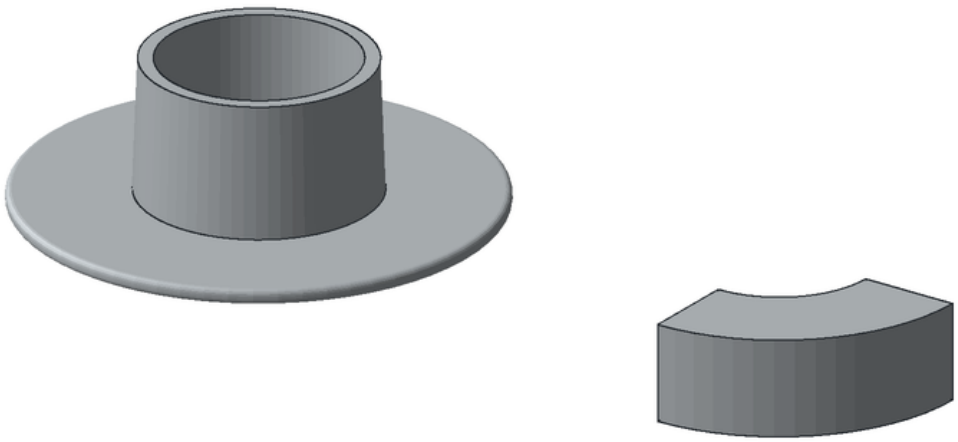
PARAMETERS

`profile`	Closed list of (R, Z) points, in any order around the loop.
`ReferenceDirection`	Where angle 0 points (perpendicular to the axis); picked automatically if left unset.
`FilletProfile corners`	Indices into the profile list; none = round every corner.

EXAMPLE

```
var file = new StepFile();
var part = file.AddPart(new StepPart("Turned", StepColor.Galvanized));
// Inlet bell: point 0 (1,0) ... point 5 (1,1.5). Round the two outside flange corners (1 and 2).
part.Add(new Revolve("Inlet bell", (0, 0, 0), (0, 0, 1),
        new (double R, double Z)[] { (1.0, 0), (2.4, 0), (2.4, 0.15), (1.2, 0.15), (1.15, 1.5), (1.0, 1.5) })
    .FilletProfile(0.06, 4, 1, 2));
part.Add(new Revolve("Quarter ring", (6, 0, 0), (0, 0, 1),
    new (double R, double Z)[] { (1, 0), (2, 0), (2, 1), (1, 1) }, angleDegrees: 90));
file.WriteStep("turned.step", "Turned");
```

RESULT



- A profile point with R = 0 lies on the axis and closes the solid there, as in a solid knob.

Sweep and Sweep.Rod - carry a section along a path

Moves a 2D section along a 3D path of points, for wire harnesses, piping, handrails and formed rod. bendRadius rounds the corners of the path.

```
new Sweep(name, profile, path, bendRadius = 0, bendSegments = 8)   .Up
Sweep.Rod(name, path, radius, bendRadius = 0, sides = 16)
```

PARAMETERS

path	Points the section's centre passes through.
Up	Direction the section's U axis points at the start of the path. Chosen automatically if left unset.

EXAMPLE

```
var file = new StepFile();
var part = file.AddPart(new StepPart("Formed", StepColor.Harness));
part.Add(Sweep.Rod("Wire", new (double, double, double)[] { (0, 0, 0.3), (5, 0, 0.3), (5, 3, 0.3), (5, 3, 3) },
    0.2, bendRadius: 0.75));
// 1 x 1/4 strap. Up = +Y keeps the 1" width along Y, so the strap bends across its thin direction.
part.Add(new Sweep("Strap", Profiles.Rectangle(0, 0, 1, 0.25),
    new (double, double, double)[] { (8, 0, 0), (8, 0, 3), (12, 0, 3), (12, 0, 0) }, 0.5) { Up = (0, 1, 0) });
file.WriteStep("formed.step", "Formed");
```

RESULT



- If a straight leg is too short for the bend radius, the radius is reduced automatically.
- A path that doubles back (a turn sharper than about 174 degrees) is rejected. Add a point or a bend radius.
- Keep the bend radius larger than half the section's size. A tighter bend makes the solid fold through itself, and nothing detects that.

Loft and Loft.SquareToRound - blend between two sections

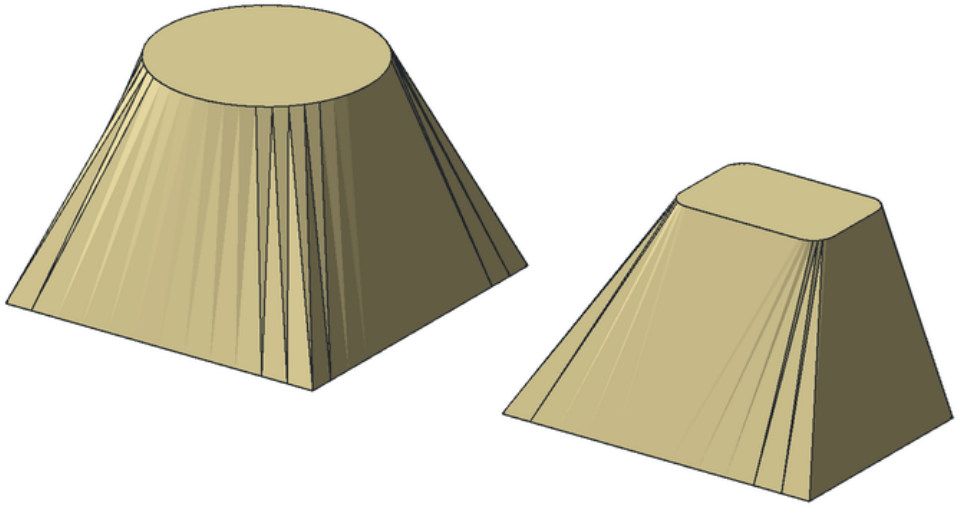
Blends a bottom section into a top section a height above it, for square-to-round transitions, reducers and offset ducts.

```
new Loft(name, origin, u, v, height, bottom, top)   .TopOffset = (du, dv)
Loft.SquareToRound(name, origin, width, depth, diameter, height, sides = 48)
```

EXAMPLE

```
var file = new StepFile();
var part = file.AddPart(new StepPart("Transitions", StepColor.Insulation));
part.Add(Loft.SquareToRound("Square to round", (0, 0, 0), 6, 6, 4, 4));
part.Add(new Loft("Offset reducer", (9, 0, 0), (1, 0, 0), (0, 1, 0), 4,
    Profiles.Rectangle(0, 0, 6, 4), Profiles.RoundedRectangle(0, 0, 3, 2, 0.4)) { TopOffset = (1, 0) });
file.WriteStep("transitions.step", "Transitions");
```

RESULT



- The two sections may have different numbers of points. Corners are kept, and the top is rotated so each bottom point pairs with the nearest top point, which avoids a twist.
- A loft is always solid. It can't be hollowed, so a sheet-metal transition can only be shown as its outer envelope, and ComputeVolume() returns the envelope's volume, not the sheet's.

MeshSolid - bring your own polyhedron

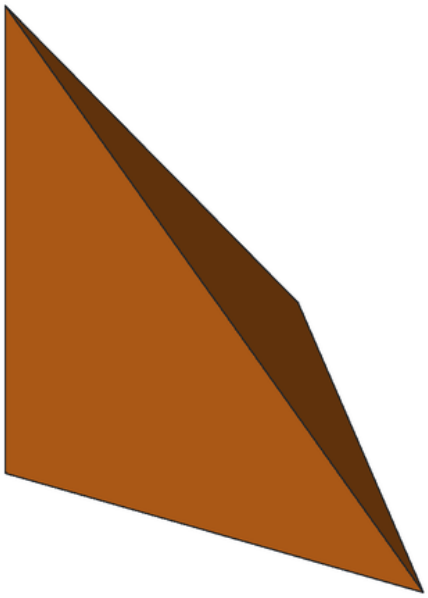
Any closed solid made of flat faces that you build yourself, for example from imported data.

```
var m = new Polyhedron();  m.AddVertex(p);  m.AddFace(int[] loop[, holes...])
new MeshSolid(name, m)
```

EXAMPLE

```
var m = new Polyhedron();
int a = m.AddVertex((0, 0, 0)), b = m.AddVertex((3, 0, 0)), c = m.AddVertex((0, 3, 0)), d = m.AddVertex((0, 0, 3));
m.AddFace(new[] { a, b, c });
m.AddFace(new[] { a, b, d });
m.AddFace(new[] { b, c, d });
m.AddFace(new[] { c, a, d });          // winding doesn't matter; it is fixed for you
var file = new StepFile();
file.AddPart(new StepPart("Wedge", StepColor.Controller)).Add(new MeshSolid("Tetrahedron", m));
file.WriteStep("tetra.step", "Wedge");
```

RESULT



- Faces must be flat and must close the solid, with every edge shared by exactly two faces. WriteStep does not reject an open or warped mesh on its own. file.Validate(), m.Validate() or StepWriteOptions.ValidateSolids = true will catch it.

Part 6 - Moving things

Each shape carries a transform that is applied when it is exported or previewed.

RotateAbout / Translate - order matters

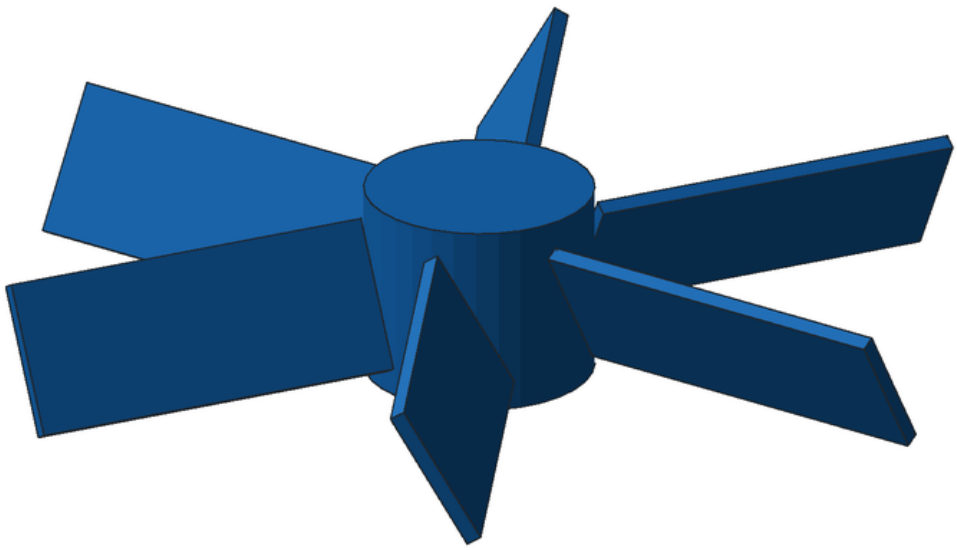
RotateAbout spins a shape about an axis line, using the right-hand rule: thumb along the axis, fingers show positive rotation. Translate moves it. Calls apply in the order you write them, and each acts on the result of the one before, in world coordinates.

```
shape.RotateAbout(point, axis, degrees)   shape.Translate(dx, dy, dz)
```

EXAMPLE

```
var file = new StepFile();
var hub = file.AddPart(new StepPart("Impeller", StepColor.Fan));
hub.Add(new Cylinder("Hub", 0, 0, 0, 0, 0, 1, 0.6, 32));
for (int i = 0; i < 6; i++)
    hub.Add(new Rectangle($"Blade {i + 1}", 1.6, 0, 0.5, 2, 0.08, 0.8)
        .RotateAbout((1.6, 0, 0.5), (1, 0, 0), 30)          // 1) pitch the blade about its own length
        .RotateAbout((0, 0, 0), (0, 0, 1), 60 * i));        // 2) then swing it round the hub
file.WriteStep("impeller.step", "Impeller");
```

RESULT



- Chaining keeps the shape's type, so new Cone(...).RotateAbout(...) is still a Cone.

MirrorAcross / TransformBy

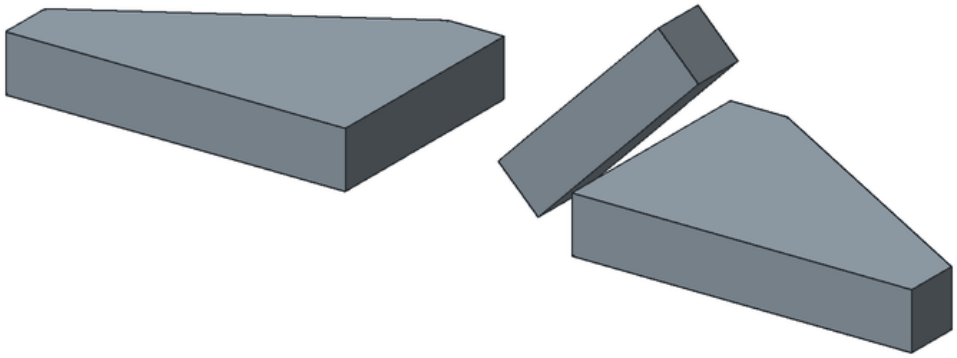
MirrorAcross reflects a shape across a plane. TransformBy applies any Transform you have built.

```
shape.MirrorAcross(point, normal)   shape.TransformBy(transform)
```

EXAMPLE

```
var file = new StepFile();
var part = file.AddPart(new StepPart("Handed", StepColor.SheetMetal));
var outline = Profiles.Polygon((0, 0), (3, 0), (3, 0.5), (0.5, 2), (0, 2));
part.Add(Extrusion.AlongZ("Right hand", 0, 0.5, outline).Translate(1, 0, 0));
part.Add(Extrusion.AlongZ("Left hand", 0, 0.5, outline).Translate(1, 0, 0)
    .MirrorAcross((0, 0, 0), (1, 0, 0)));                    // mirror across the YZ plane
part.Add(new Rectangle("Spacer", 0, 1, 1, 0.5, 0.5, 2)
    .TransformBy(Transform.RotationAbout((0, 1, 0), (0, 1, 0), 45)));
file.WriteStep("handed.step", "Handed");
```

RESULT



- A mirrored solid is re-wound automatically, so it is still valid (not inside out).

Moving a whole part

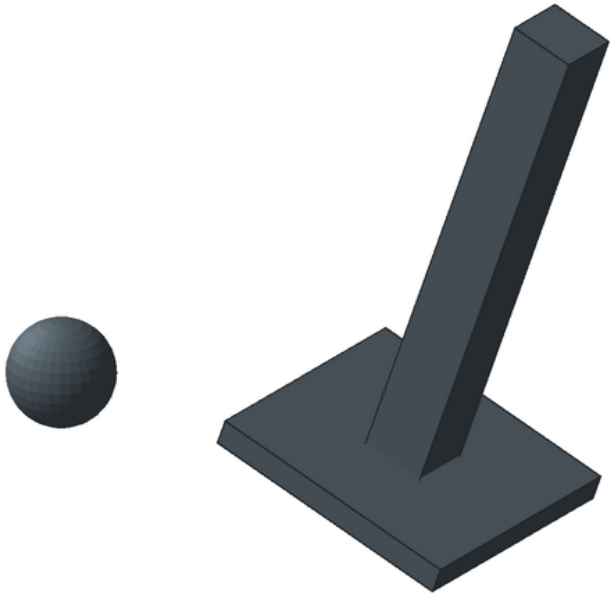
StepPart has the same methods: RotateAbout, Translate, MirrorAcross and TransformBy. They move the shapes that are in the part at that moment.

```
part.RotateAbout(...)   part.Translate(...)   part.MirrorAcross(...)   part.TransformBy(...)
```

EXAMPLE

```
var file = new StepFile();
var leg = file.AddPart(new StepPart("Leg", StepColor.Channel));
leg.Add(new Rectangle("Post", 0, 0, 2, 0.5, 0.5, 4));
leg.Add(new Rectangle("Foot", 0, 0, 0.1, 2, 2, 0.2));
leg.RotateAbout((0, 0, 0), (0, 1, 0), 20).Translate(3, 0, 0);    // add shapes FIRST, then move
leg.Add(new Sphere("Not moved", 0, 0, 0, 0.4));                  // added afterwards: stays put
file.WriteStep("leg.step", "Leg");
```

RESULT



- Add every shape before you move the part. A shape added later stays where it was drawn, like the sphere at the origin in the picture.

Transform - building transforms yourself

Transform values combine with Then(...) in reading order. Use them with TransformBy, and as instance placements (Part 7).

```
Transform.Identity   Translation(dx, dy, dz)   RotationAbout(point, axis, deg)
Mirror(point, normal)   Scale(factor[, centre])   Placement(origin, xAxis, zAxis)
FromBasis(centre, ax, ay, az)   a.Then(b)   t.Inverse()   t.Apply(p)   t.ApplyDirection(d)
t.IsRigid   t.IsIdentity   t.Determinant
```

EXAMPLE

```
var t = Transform.RotationAbout((0, 0, 0), (0, 0, 1), 90)        // first rotate...
    .Then(Transform.Translation(10, 0, 0));                        // ...then move
Console.WriteLine(t.Apply((1, 0, 0)));                             // (10, 1, 0)
Console.WriteLine(t.Inverse().Apply((10, 1, 0)));                  // back to (1, 0, 0)
var frame = Transform.Placement((5, 5, 0), (0, 1, 0), (1, 0, 0));  // local Z -> world +X
Console.WriteLine($"rigid: {frame.IsRigid}, scale rigid: {Transform.Scale(2).IsRigid}");
```

OUTPUT

```
(10, 1, 0)
(1, 0, 0)
rigid: True, scale rigid: False
```

- Printed values can show tiny rounding errors such as 6.1E-17; treat those as zero.

Part 7 - Colours, assemblies and instances

StepColor and WithColor

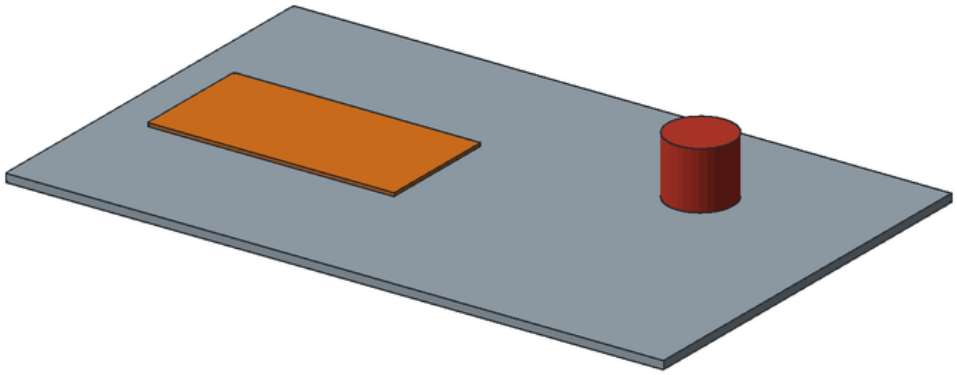
Colours are written into the STEP file, and the WPF preview uses them too. Give a whole part a colour, or override it on single shapes.

```
new StepPart(name, color)   shape.WithColor(color)
StepColor.FromHex("#RRGGBB")   new StepColor(r, g, b)   presets: SheetMetal, Galvanized, Fan,
Controller, Channel, Mount, Harness, Border, Insulation, Rubber
```

EXAMPLE

```
var file = new StepFile();
var part = file.AddPart(new StepPart("Panel", StepColor.SheetMetal));      // part colour
part.Add(new Rectangle("Panel", 0, 0, 0.05, 8, 5, 0.1));
part.Add(new Cylinder("Knob", 2, 1, 0.1, 2, 1, 0.8, 0.4, 32)).WithColor(StepColor.FromHex("#C0392B"));
part.Add(new Rectangle("Label", -2, 0, 0.12, 3, 1.5, 0.04)).WithColor(StepColor.Controller);
file.WriteStep("colours.step", "Panel");
```

RESULT



- Precedence: shape colour, then part colour, then StepWriteOptions.DefaultColor (uncoloured if that is null). The WPF preview falls back to sheet-metal grey instead of uncoloured.

AddInstance - one part, many copies

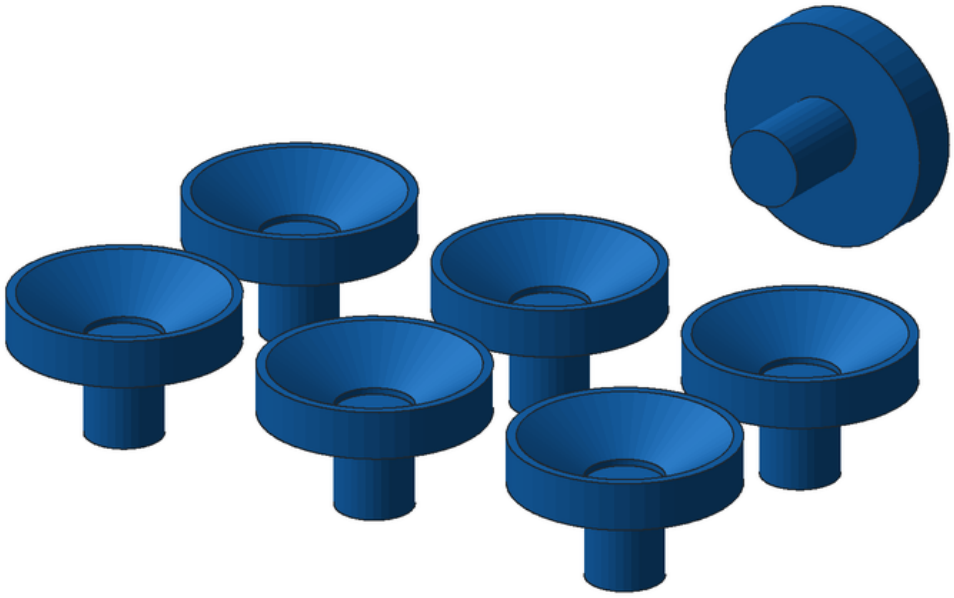
Draw a part once around its own origin, then place copies of it. The geometry is written once and referenced by each copy. Files stay small, and CAD counts the copies as one part in the bill of materials.

```
file.AddInstance(part, dx, dy, dz[, name])
file.AddInstance(part, placement[, name])
```

EXAMPLE

```
var file = new StepFile();
var fan = new StepPart("Plug fan", StepColor.Fan);                  // not AddPart: only the copies appear
fan.Add(new Cylinder("Motor", 0, 0, 0, 0, 0, 3, 1.2, 32));
fan.Add(new Revolve("Wheel", (0, 0, 3), (0, 0, 1),
    new (double R, double Z)[] { (1.2, 0), (3.5, 0), (3.5, 1.5), (3.2, 1.5), (1.2, 0.2) }, 360, 48));
for (int row = 0; row < 2; row++)
    for (int col = 0; col < 3; col++)
        file.AddInstance(fan, col * 9, row * 9, 0, $"Fan {row * 3 + col + 1}");
file.AddInstance(fan, Transform.RotationAbout((0, 0, 0), (1, 0, 0), -90)
    .Then(Transform.Translation(9, 20, 3.5)), "Spare (on its side)");
file.WriteStep("array.step", "Fan array");
```

RESULT



- Placements must be rigid: rotation and translation only. A scale or mirror throws at WriteStep, because STEP assemblies can't scale or mirror a copy. Transform the shapes instead. The WPF preview does not reject them, so check with Validate().
- If the part is also added with AddPart, it appears once more at its drawn position.

Part 8 - Checking, measuring and handling errors

When an error is reported

Most impossible geometry throws an exception rather than writing a broken file. Some problems are only found when you ask for a check, and self-intersection is never found. Some checks run when you call a method; most run when the shape is built.

EXAMPLE

```
var plate = Extrusion.AlongZ("Plate", 0, 0.25, Profiles.Rectangle(0, 0, 4, 4)).AddCircularCut(9, 0, 1);
Console.WriteLine("AddCircularCut accepted the bad hole - nothing is checked yet");
try { plate.ToPolyhedron(); }
catch (ArgumentException e) { Console.WriteLine("Building it: " + e.Message); }

try { Extrusion.AlongZ("Strip", 0, 1, Profiles.Rectangle(0, 0, 4, 0.5)).Shell(0.3, 0.1); }
catch (ArgumentException e) { Console.WriteLine("Shell checks immediately: " + e.Message.Split('(')[0]); }
```

OUTPUT

```
AddCircularCut accepted the bad hole - nothing is checked yet
Building it: Plate: cut 0 runs outside the outline or too close to its edge.
Shell checks immediately: Strip shell: an offset of 0.3 is larger than the profile allows
```

Checked immediately (throws at the call)	Shell and AddPocketGrid offset errors (wall too thick for the outline), AddPocketGrid convexity and 'no room for ribs', Profiles.Offset, Profiles.ThickPolyline; AddPocketGrid on an extrusion with no profile yet
Checked when built (listed by Validate(); throws at WriteStep and in the WPF preview)	Cuts and pockets inside the outline and not overlapping, pocket depths (including a Shell or AddPocketGrid floor that is 0 or too deep), fillet/chamfer sizes, self-crossing outlines, sweeps that double back, torus/tube radii
Checked by Validate() and on request	Solid checks for every shape (closed, consistently oriented, flat faces): run by file.Validate(), StepWriteOptions.ValidateSolids or Polyhedron.Validate(). WriteStep alone doesn't run them, which matters mostly for MeshSolid
Not detected	Solids that pass through themselves (very tight sweeps, thin shells on steep draft) and shapes that overlap each other
Silently skipped on export	Zero-size shapes; Validate() still lists them

SkipInvalidShapes - export what you can

For exports triggered from a UI, writing a partial file plus a warning is often better than failing. SkipInvalidShapes leaves out the bad shapes and lists them.

EXAMPLE

```
var file = new StepFile();
var door = file.AddPart("Door");
door.Add(Extrusion.AlongZ("Skin", 0, 0.05, Profiles.Rectangle(0, 0, 10, 10)).AddCircularCut(20, 0, 1));
door.Add(new Rectangle("Frame", 0, 0, 1, 10, 10, 0.5));
var options = new StepWriteOptions { SkipInvalidShapes = true };
file.WriteStep("door.step", "Door", options);
foreach (var s in options.SkippedShapes) Console.WriteLine("Skipped: " + s);
```

OUTPUT

```
Skipped: Door: Skin: cut 0 runs outside the outline or too close to its edge.
```

- The WPF preview has no skip option, and a bad shape there throws. In a UI, run file.Validate() first and show the problems before you build the preview.

ComputeVolume / Bounds / ToPolyhedron

Measure what you built, for example for weights or space claims.

> **`shape.ComputeVolume()` `part.ComputeVolume()` `file.Bounds()` `shape.ToPolyhedron()`**
> **`Polyhedron: Volume, SurfaceArea, Min, Max, Vertices, Faces, Validate()`**

EXAMPLE

```
var file = new StepFile();
var plate = file.AddPart("Plate");
plate.Add(Extrusion.AlongZ("p", 0, 0.125, Profiles.Rectangle(0, 0, 24, 24)).AddCircularCut(0, 0, 9.5, 128));
double cubicInches = plate.ComputeVolume();
Console.WriteLine($"Volume {cubicInches:F2} in^3, steel weight {cubicInches * 0.284:F2} lb");
var (min, max) = file.Bounds();
Console.WriteLine($"Bounds {min} to {max}");
var solid = plate.shapes[0].ToPolyhedron();
Console.WriteLine($"{solid.Faces.Count} faces, area {solid.SurfaceArea:F1} in^2, problems: {solid.Validate().Count}");
```

OUTPUT

```
Volume 36.57 in^3, steel weight 10.39 lb
Bounds (-12, -12, 0) to (12, 12, 0.125)
134 faces, area 604.6 in^2, problems: 0
```

- Each call to ComputeVolume, Validate, WriteStep or the WPF Build rebuilds the geometry. Keep the result of ToPolyhedron() if you need it more than once.

Part 9 - Previewing in WPF (QPAC.Sculptor.Wpf)

The WPF package shows the same solids in a HelixToolkit viewport. Add the QPAC.Sculptor.Wpf package, also reference QPAC.Sculptor 1.1.1 or later directly (it fixes preview triangulation of grid-aligned holes; the WPF package alone may resolve 1.1.0), and put a HelixViewport3D named viewport in your window. The examples in this part are compiled and run in a WPF test program. They show no pictures because their output is a live 3D view.

SteptoVisual3D.Build

Turns a StepFile into visuals you add to the viewport: one SculptorVisual3D per part and per instance. With ShowEdges on, it also returns outline (LinesVisual3D) visuals.

EXAMPLE (WPF)

```
using HelixToolkit.Wpf;
using QPAC.Sculptor;
using QPAC.Sculptor.Wpf;

var stepFile = new StepFile();
stepFile.AddPart(new StepPart("Box", StepColor.Fan)).Add(new Rectangle("b", 0, 0, 0, 2, 2, 2));
viewport.Children.Add(new DefaultLights());
foreach (var visual in SteptoVisual3D.Build(stepFile))
    viewport.Children.Add(visual);
viewport.ZoomExtents();
```

SculptorRenderOptions

Controls shading, outlines and fallback colours.

PARAMETERS

`SmoothShading / CreaseAngleDegrees`	Blend facets that meet at less than the angle (default on, 30 degrees).
`ShowEdges / EdgeColor / EdgeThickness`	Outline sharp edges.
`TypeColors`	Colour by StepPart.Type when the part has no colour.
`DefaultColor`	Last fallback (sheet-metal grey).
`Opacity, SpecularPower`	Material look.

EXAMPLE (WPF)

```
using System.Windows.Media;
using QPAC.Sculptor;
using QPAC.Sculptor.Wpf;

var stepFile = new StepFile();
stepFile.AddPart(new StepPart("Fan unit") { Type = "Fan" }).Add(new Cylinder("hub", 0, 0, 0, 0, 0, 1, 1, 48));
var options = new SculptorRenderOptions { ShowEdges = true, EdgeColor = Colors.Black };
options.TypeColors["Fan"] = Colors.SteelBlue;
foreach (var visual in SteptoVisual3D.Build(stepFile, options))
    viewport.Children.Add(visual);
```

- Colour order in the preview: shape, part, TypeColors[part.Type], DefaultColor.

Picking a part with the mouse

Every part visual is a SculptorVisual3D, which knows its Part, its Instance and its Name.

EXAMPLE (WPF)

```
using System.Windows;
using System.Windows.Input;
using HelixToolkit.Wpf;
using QPAC.Sculptor.Wpf;

viewport.MouseDown += (s, e) =>
{
    var hit = viewport.Viewport.FindHits(e.GetPosition(viewport)).FirstOrDefault();
    if (hit?.Visual is SculptorVisual3D picked)
        MessageBox.Show($"Clicked {picked.Name}");
};
```

BuildModel / BuildPart / ToMesh and threading

Other entry points for special cases:

SteptoVisual3D.BuildModel(file, options)	One frozen Model3DGroup for the whole model. The fastest option; no per-part picking and no outlines.
SteptoVisual3D.BuildPart(part, options)	The visual for one part, e.g. after it changed. It does not update that part's instance visuals or outlines.
SteptoVisual3D.BuildPartModel(part, options)	One part as a frozen Model3DGroup.
SteptoVisual3D.ToMesh(shape, creaseAngle)	A frozen MeshGeometry3D; creaseAngle 0 = flat shading.
ToColor / ToStepColor	Convert between StepColor and WPF Color.

- Threading: call Build and BuildPart on the UI thread. To keep the UI responsive with big models, call BuildModel or BuildPartModel on a background thread (the results are frozen), then put the result into a ModelVisual3D on the UI thread.
- Only rebuild what changed. Tessellating is the expensive step, and lower sides/segments values preview faster.

Part 10 - Putting it together

A complete fan-wall cell built from scratch: a frame of formed channel, a fan plate with a bolt pattern, a blank-off, a fan used three times through instances, a wire harness, and a check before export.

Worked example: fan-wall cell

EXAMPLE

```
var file = new StepFile { Description = "Fan wall cell - walkthrough example" };
double W = 40, H = 28, t = 0.0625;                      // cell size and sheet gauge

// 1. Frame: four rails of 2" channel, flanges toward the front (+Z).
var frame = file.AddPart(new StepPart("Frame", StepColor.Channel));
var channel = Profiles.Channel(2, 1, t, t);
frame.Add(new Extrusion("Bottom rail", (0, 1, 0), (0, 1, 0), (0, 0, 1), W).SetProfile(channel));
frame.Add(new Extrusion("Top rail", (0, H - 1, 0), (0, 1, 0), (0, 0, 1), W).SetProfile(channel));
frame.Add(new Extrusion("Left rail", (1, 2, 0), (-1, 0, 0), (0, 0, 1), H - 4).SetProfile(channel));
frame.Add(new Extrusion("Right rail", (W - 1, 2, 0), (-1, 0, 0), (0, 0, 1), H - 4).SetProfile(channel));

// 2. Fan plate (left half) and blank-off (right half) sitting on the rail webs.
var plate = file.AddPart(new StepPart("Fan plate", StepColor.Galvanized));
plate.Add(Extrusion.AlongZ("Plate", -t, t, Profiles.Rectangle(W / 4, H / 2, W / 2, H))
    .AddCircularCut(W / 4, H / 2, 8, 128)
    .AddBoltCircle(W / 4, H / 2, 9, 8, 0.15, 22.5)
    .AddCircularCut(W / 4, 3, 0.35, 24));                // grommet hole for the motor lead
file.AddPart(new StepPart("Blank-off", StepColor.SheetMetal))
    .Add(Extrusion.AlongZ("Blank", -t, t, Profiles.Rectangle(3 * W / 4, H / 2, W / 2, H)));

// 3. One fan definition, placed behind the plate opening (and two spares beside the cell).
var fan = new StepPart("Plug fan", StepColor.Fan);
fan.Add(new Cylinder("Motor", 0, 0, -9, 0, 0, -3, 2.5, 32));
fan.Add(new Revolve("Wheel", (0, 0, -3), (0, 0, 1),
    new (double R, double Z)[] { (2.5, 0), (7.5, 0), (7.5, 2.5), (6.8, 2.5), (2.5, 0.3) }, 360, 64));
file.AddInstance(fan, W / 4, H / 2, 0, "Fan 1");
file.AddInstance(fan, W + 12, 8, 0, "Spare A");
file.AddInstance(fan, W + 12, 25, 0, "Spare B");

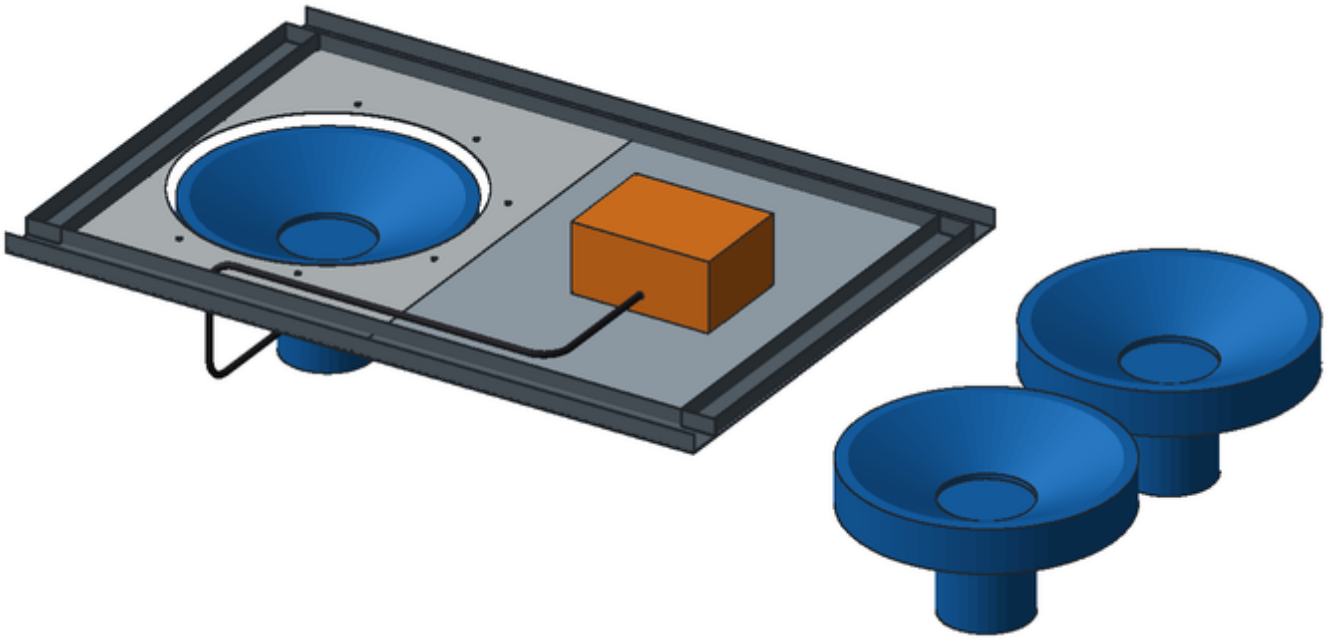
// 4. Harness along the rails to the controller box.
file.AddPart(new StepPart("Controller", StepColor.Controller)).Add(new Rectangle("Box", 3 * W / 4, H / 2, 2, 8, 6, 4));
var leadPath = new (double, double, double)[]
{
    (W / 4, H / 2 - 2.5, -6),        // leaves the motor
    (W / 4, 3, -6), (W / 4, 3, 1),   // down, then forward through the grommet
    (3 * W / 4, 3, 1),               // along the bottom rail
    (3 * W / 4, H / 2 - 3, 1),       // up into the controller
};
file.AddPart(new StepPart("Harness", StepColor.Harness)).Add(Sweep.Rod("Motor lead", leadPath, 0.2, bendRadius: 1.5));

// 5. Check, then write.
var problems = file.Validate();
Console.WriteLine(problems.Count == 0 ? "No problems" : string.Join("\n", problems));
file.WriteStep("fan-cell.step", "Fan wall cell");
```

OUTPUT

```
No problems
```

RESULT



Troubleshooting

NU1101 / NU1301: cannot find QPAC.Sculptor	The qpacsystems feed isn't configured, or its token expired (Step 0).
A new version is published but the app builds the old one	The reference is 1.0.*, or NuGet cached a republished version: delete %USERPROFILE%\.nuget\packages\qpac.sculptor\<version>.
FileNotFoundException: Could not load 'QPAC Sculptor, Version=1.0.x.x'	A package was built with a different AssemblyVersion. It must stay 1.0.0.0 for every 1.x release.
ArgumentException naming a shape	That shape's geometry is impossible; the message says why. Run file.Validate() to see every problem at once.
CAD shows the model 25.4 times too big or too small	StepFile.Units doesn't match the unit you drew in. Set Units to your drawing unit; use ExportUnits to convert.
Holes look like polygons in CAD	Curves are faceted. Raise sides on cuts and cylinders.
Shapes added after part.RotateAbout(...) didn't move	Part transforms apply only to shapes already in the part (Part 6).
A tray comes out upside down	Shell's openFront defaults to true, and with AlongZ the front face is the bottom. Pass openFront: false for a tray that opens upward.
CI fails with 401/403 on the qpacsystems feed	The QPAC_PACKAGES_PAT secret is missing or expired. Update it in both repositories. In the WPF repo the failure shows at 'Add qpacsystems feed' or 'Restore', on pull requests too.

Review findings and suggestions

Before publishing, this walkthrough was reviewed against the library source code by a second reviewer. Every factual problem found was corrected above. These are the issues the review turned up with the library itself and with how this guide approaches it, most important first.

1. Geometry is faceted, not exact

Holes, cylinders and fillets are written as flat facets, so CAD users can't measure a hole diameter or pick a true cylindrical face, and files grow large: the full gallery is 6 MB. Writing true analytic surfaces for the common cases (cylinders, holes, cones, planar circles) would make the output proper CAD geometry. This is the biggest gap.

2. No way to subtract one solid from another

There are no boolean operations (union or subtract between shapes). Cuts and pockets cover plates and blocks, but hollow lofts, off-axis holes through revolved parts and notched channels can't be modelled. Simple booleans would cover these, and the loft limitation in Part 5 goes away with them.

3. Validate() and the exporter disagree on zero-size shapes

WriteStep skips them silently, but Validate() reports them. A configurator that requires an empty Validate() will fail on intentional zero sizes. Validate should skip empty solids, as the exporter does.

4. Some errors are not detected

Self-intersecting solids (tight sweep bends, thin shells on drafted parts) pass every check. Draft doesn't reach pockets or Shell interiors, so the shell wall can break through the tapered side. Shell should follow the draft, and Validate should test for self-intersection.

5. Error timing is inconsistent

Shell and AddPocketGrid throw at the call, while AddCut and AddPocket throw only when the shape is built. Pick one; checking at build time everywhere is friendlier for configurators, and errors should then go through Validate() consistently.

6. Preview and export can differ

The WPF preview accepts non-rigid instance placements that WriteStep rejects. It colours uncoloured shapes grey where STEP leaves them uncoloured, and it has no SkipInvalidShapes. Sharing one rule set between the two would make 'the preview matches the file' strictly true.

7. Geometry is rebuilt on every call

Validate, WriteStep, ComputeVolume and each WPF Build all re-tessellate every shape, and ShowEdges does it again per instance. Caching the polyhedron per shape (cleared when the shape changes) would make big SPFA models much faster.

8. Legacy names and defaults are confusing

Extrusion.AddCircle sets the outline rather than adding a hole. Default facet counts vary (AddCircularCut 256, Profiles.Circle 64, Cylinder 16). Shell's openFront default produces upside-down trays with AlongZ. New, clearer names (SetCircle, and an opening-side enum) could be added while the old ones are kept, and the defaults made consistent.

9. StepWriteOptions.SkippedShapes never resets

Reusing one options object accumulates results from earlier exports. Clear the list at the start of each write, or return the list from WriteStep.

10. Releases can be missed

CI publishes the version in the csproj and skips versions that already exist. A fix merged without a version bump therefore never ships, with no warning. Also, a manual run publishes from any branch. CI should fail when the code changed but the version didn't, and should publish only from master.

11. Ownership of publishing

Both repositories sit under a personal GitHub account and publish with one personal token. Today the CI secret holds the same token as the author's local NuGet configuration, so when it expires CI and the author's local restores break together. Moving the repositories into the qpacsystems organisation lets CI use the built-in token.

12. About this guide

This document explains each function separately. That is good for reference, but new users would also benefit from 2-3 short task-based tutorials, for example 'model a door panel' and 'export a configured SPFA', built from real Q-PAC parts. Parameter tables are written by hand, so they can drift from the code when signatures change. The examples can't drift, because they are compiled when the PDF is built, but generating the tables from the XML documentation would close that gap too.